Computer Organisation

Topic 7
Central Processing Unit

Stallings, Computer Organization & Architecture (8th Ed.), Chap. 12 & 3

Hamacher et al., Computer Organisation (5th Ed.), Chap. 7 & 8

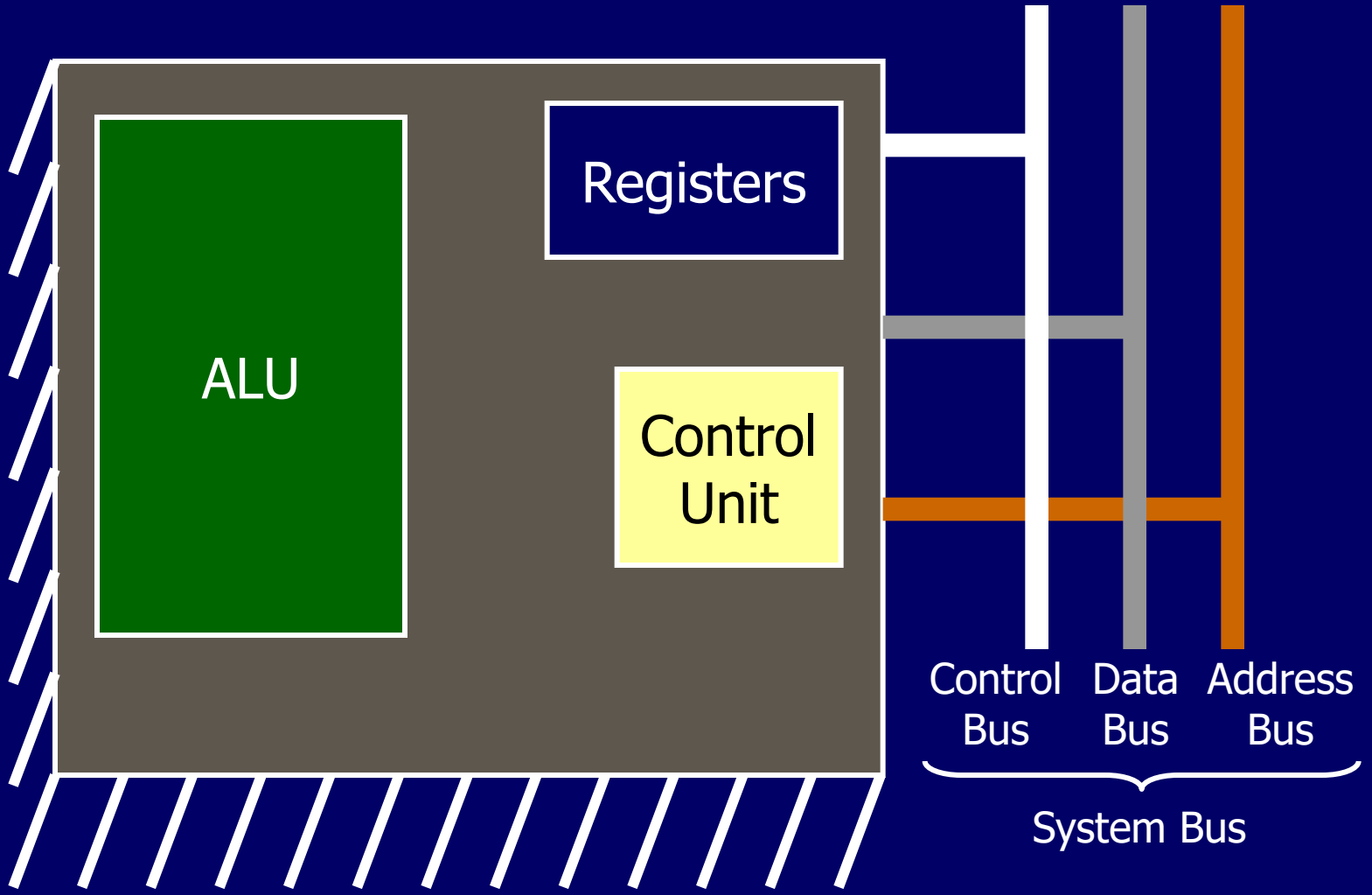
Outline

- Processor & Register Organisation
  - CPU structure
  - User-visible registers
  - Control & status registers
- Instruction Cycle
  - Fetch, execute, interrupt & indirect cycle
  - Data flow
- Instruction Pipelining
  - Performance
  - Branch

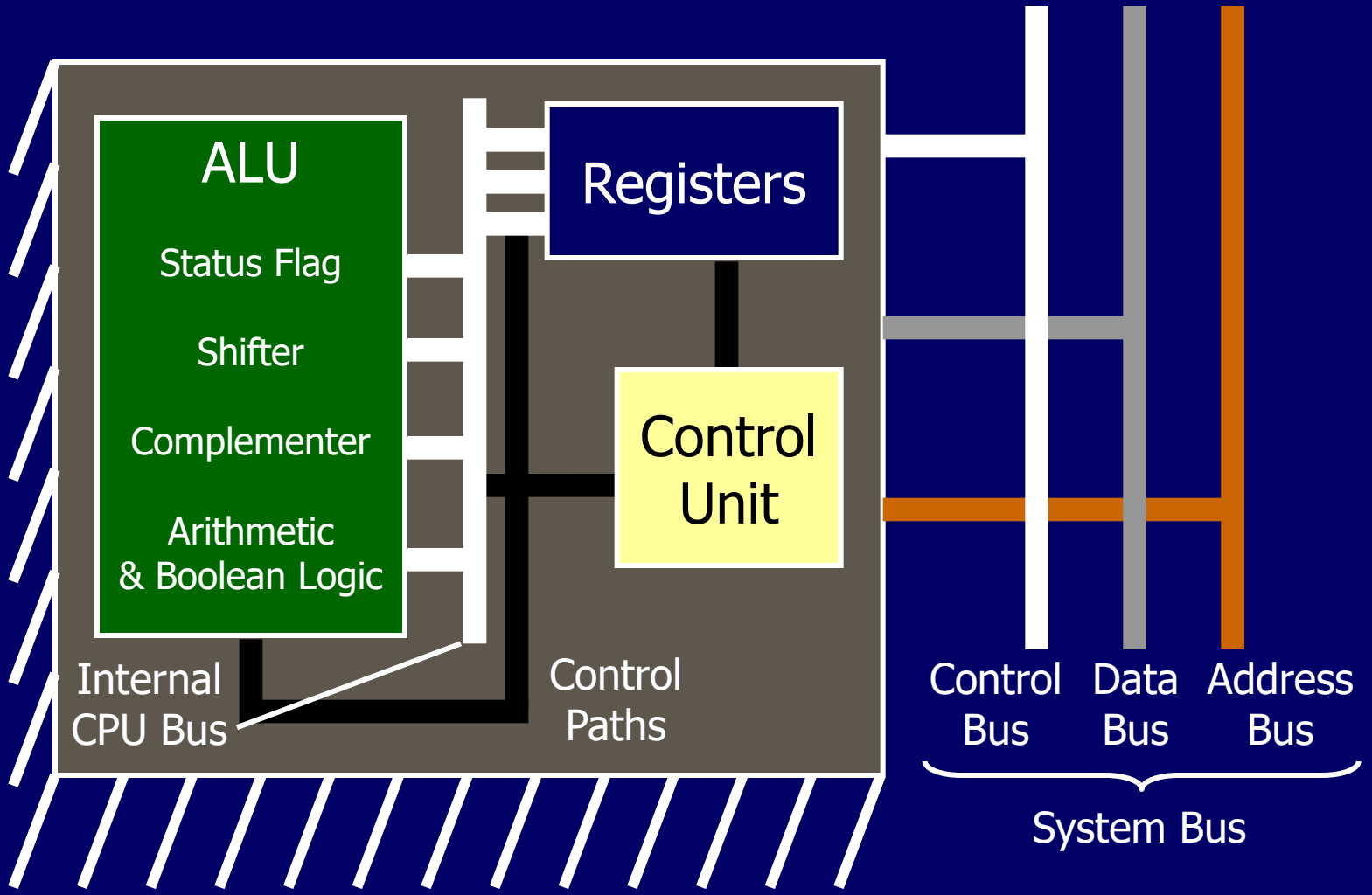
CPU Structure (1)

- CPU must:
  - Fetch instructions (from memory)
  - Interpret instructions (decode)
  - Fetch data (from memory or IO module)
  - Process data (arithmetic or logic)
  - Write data (to memory or IO module)
- CPU components:
  - Arithmetic and Logic Unit (ALU) - does computation
  - Control Unit (CU) - controls data/instruction movement
  - Registers - internal memory

CPU Structure (2)
CPU with Systems Bus

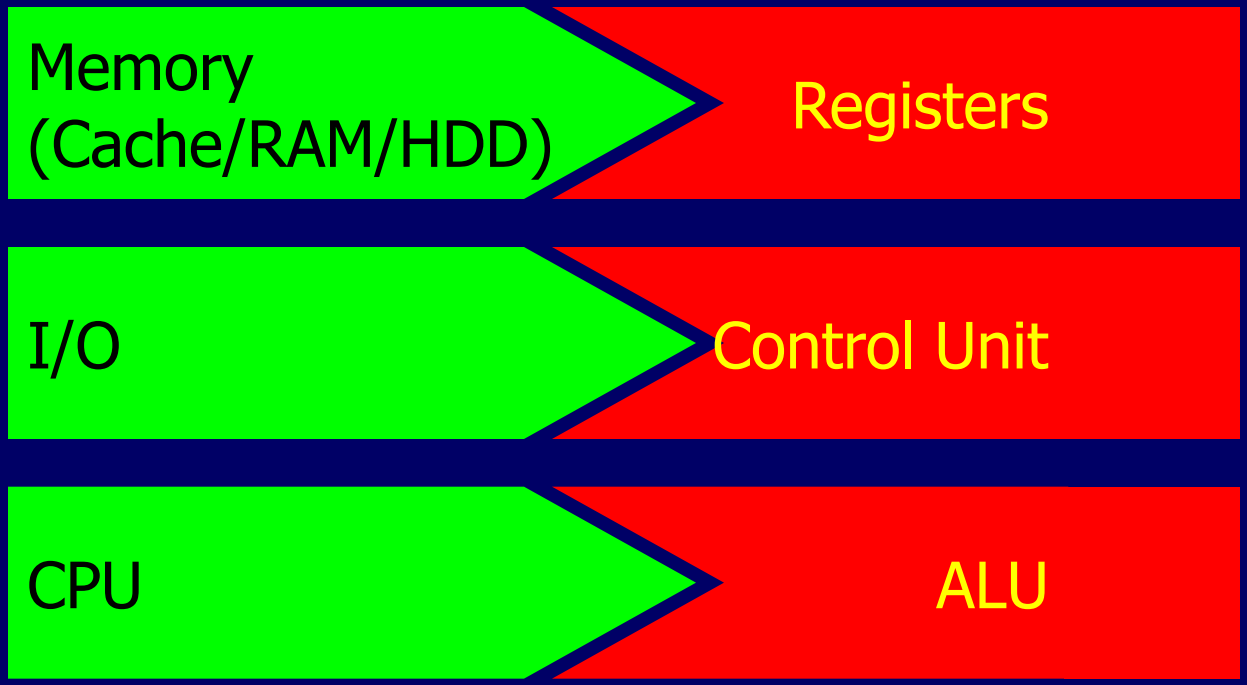


CPU Structure (3)
Internal Structure of CPU



CPU Structure (4)

- Internal structure of PC ≡ internal structure of CPU



Registers

- CPU must have some working space (temporary storage) - called Registers
- Number and function vary between processor designs
- One of the major design decisions
- Top level of memory hierarchy (above cache and main memory)
- 2 functions:
  - User-visible registers
  - Control and status registers

User Visible Registers

- Can be referenced by assembly-level instructions
- Categories of user-visible registers:
  - General Purpose
    - Data
    - Address

    } may be considered general purpose
  - Condition Codes

General Purpose Registers (1)

- May be true general purpose
- May be restricted
- May be used for data or addressing (e.g. register indirect, displacement)
- **Data register**
  - Hold data only (accumulator) - cannot use for calculating operand address
- **Address register**
  - Segment pointer (base address of segment)
  - Index register (for indexed addressing, autoindexed)
  - Stack pointer (allows implicit addressing)

General Purpose Registers (2)

- Design issues:
  - True general-purpose or specialised?
  - How many?
  - Register size?

General Purpose Registers (3)
General Purpose or Specialised

- Make them general purpose
  - Increase flexibility and programmer options
  - Increase instruction size (need more bits to specify purpose) & complexity
- Make them specialised
  - Implicit reference to a register type (in opcode)
  - Smaller (faster) instructions
  - Less flexibility

General Purpose Registers (4) How Many GP Registers?

- Between 8 - 32
- Fewer = more memory references
- More does not reduce memory references and takes up processor resource
- RISC (finds advantage in using hundreds of registers)

General Purpose Registers (5) How big?

- Large enough to hold the largest address (for address registers)
- Large enough to hold most data types (for data registers)
- Often possible to combine two data registers
  - e.g. in C programming with
    - double int a;
    - long int a;
- Some machines allow 2 contiguous registers to hold double-length values

Condition Code Registers

- Also called flags
- Sets of ***individual*** bits
  - e.g. result of last operation was zero
- Can be read (implicitly) by programs
  - e.g. Jump if zero
- Can not (usually) be set by programs
- In some machines, procedure call will cause all user-visible registers to be saved (procedure can now use these registers)

Condition Code Registers

- … like lights on a control panel

Image Source: https://www.flickr.com/photos/telstar/2861932210/

Control & Status Registers (1)

- Not visible to users (on most machines)
- Used to control operation of CPU
- 4 main registers for instruction execution:
  - Program Counter (PC) - contains address of instruction to be fetched
    - updated by CPU after instruction fetch
    - always points to next instruction
    - branch/skip instruction also modifies PC
  - Instruction Register (IR) - contains fetched instruction
    - opcode and operand specifiers are analysed

Control & Status Registers (2)

- 4 main registers for instruction execution (cont):
  - Memory Address Register (MAR) - contains address of a location in memory (connects to address bus)
  - Memory Buffer Register (MBR) - contains data to be written to memory or fetched from memory (connected to data bus)
    - user-visible registers exchange data with MBR
- CPU may have direct access to MBR and user-visible registers

Control & Status Registers

- … like information display

Image Source: https://www.flickr.com/photos/zeusandhera/408799829

Program Status Word

- A set of registers containing status information
- Includes condition codes (flags)
- Common flags:
  - Sign: sign bit of last arithmetic operation
  - Zero: if result = 0
  - Carry: if result involves ‘carry’ or ‘borrow’ of bits
  - Equal: for logical compare
  - Overflow: if arithmetic overflow
  - Interrupt enable/disable
  - Supervisor: if CPU executing in supervisor/user mode

Supervisor Mode

- Intel ring zero
- Kernel mode
- Allows privileged instructions to execute
- Used by operating system
- Not available to user programs
- Instructions that operate on status register or may affect normal state of processor (http://www.atarimagazines.com/startv1n3/SupervisorMode.html)

Other Registers

- May have registers pointing to:
  - Process control blocks (see OS)
  - Interrupt vectors (see OS)
  - Stacks (stack pointer for procedure call)
  - Page table (for virtual memory systems)
- CPU design (including design of control and status register organization) and operating system design are closely linked
- Control information between registers and memory also a register design issue (first few hundred/thousand words of memory for control)

Motorola MC68000

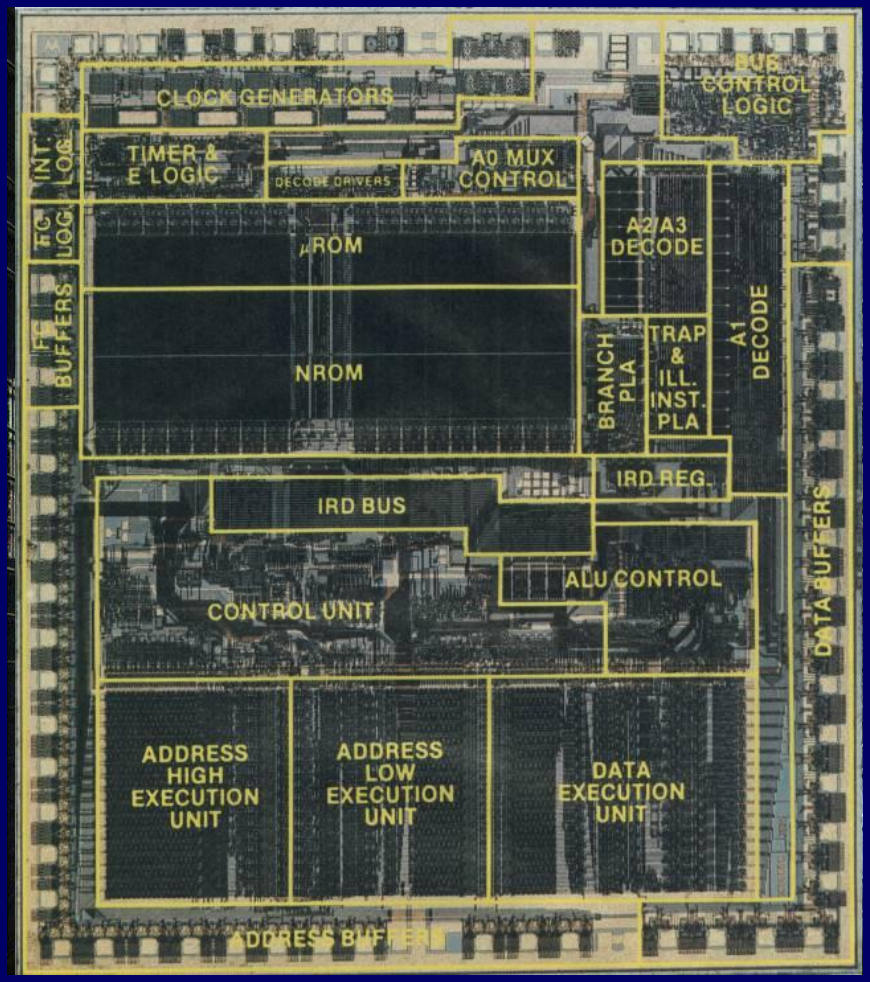


Image Source: https://en.wikipedia.org/wiki/Motorola_68000; http://www.easy68k.com/paulrsm/doc/dpbm68k2.htm

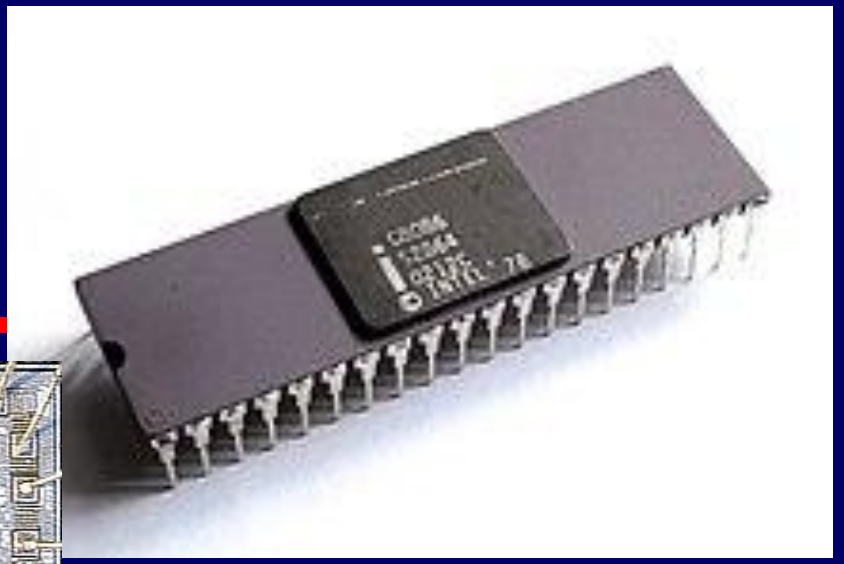
Motorola MC68000 Register Organisation (1)

- Has 32-bit registers
- 8 data registers
  - for data manipulation
  - as index registers for addressing
  - register width allows 8-, 16-, & 32-bit data operations
- 9 address registers
  - contain 32-bit addresses
  - 2 stack pointers
- 32-bit program counter (PC) and 16-bit status register

Motorola MC68000 Register Organisation (2)

- Motorola wanted very regular instruction set
- No special purpose register

Intel 8086



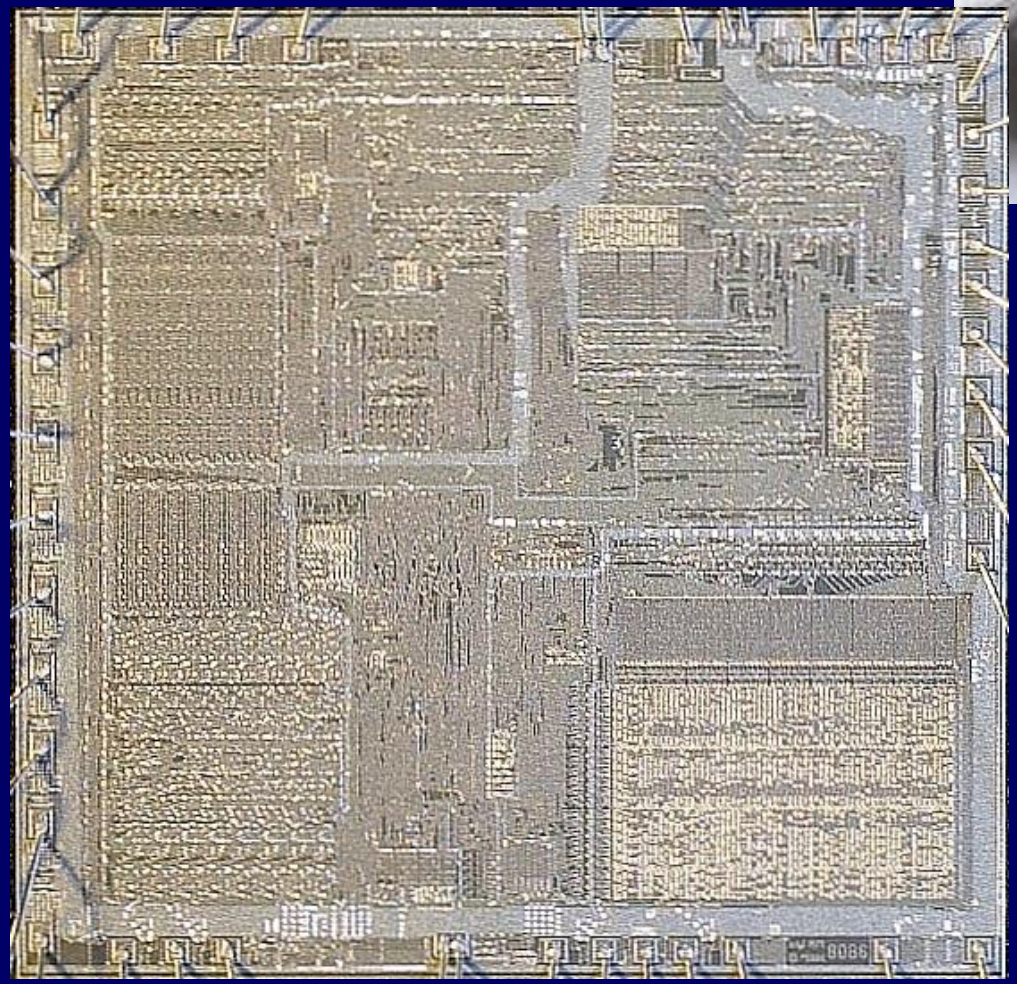


Image Source: https://en.wikipedia.org/wiki/Intel_8086

Intel 8086 Register Organisation (1)

- Every register is special-purpose (some can be used as general-purpose)
- 4 16-bit data registers
  - AC, Base, Count & Data registers
  - can be general purpose
  - sometimes used implicitly, e.g. the AC
- 4 16-bit pointer and index registers
  - Stack pointer, Base pointer, Source index & Dest index
  - sometimes used implicitly, e.g. segment offset

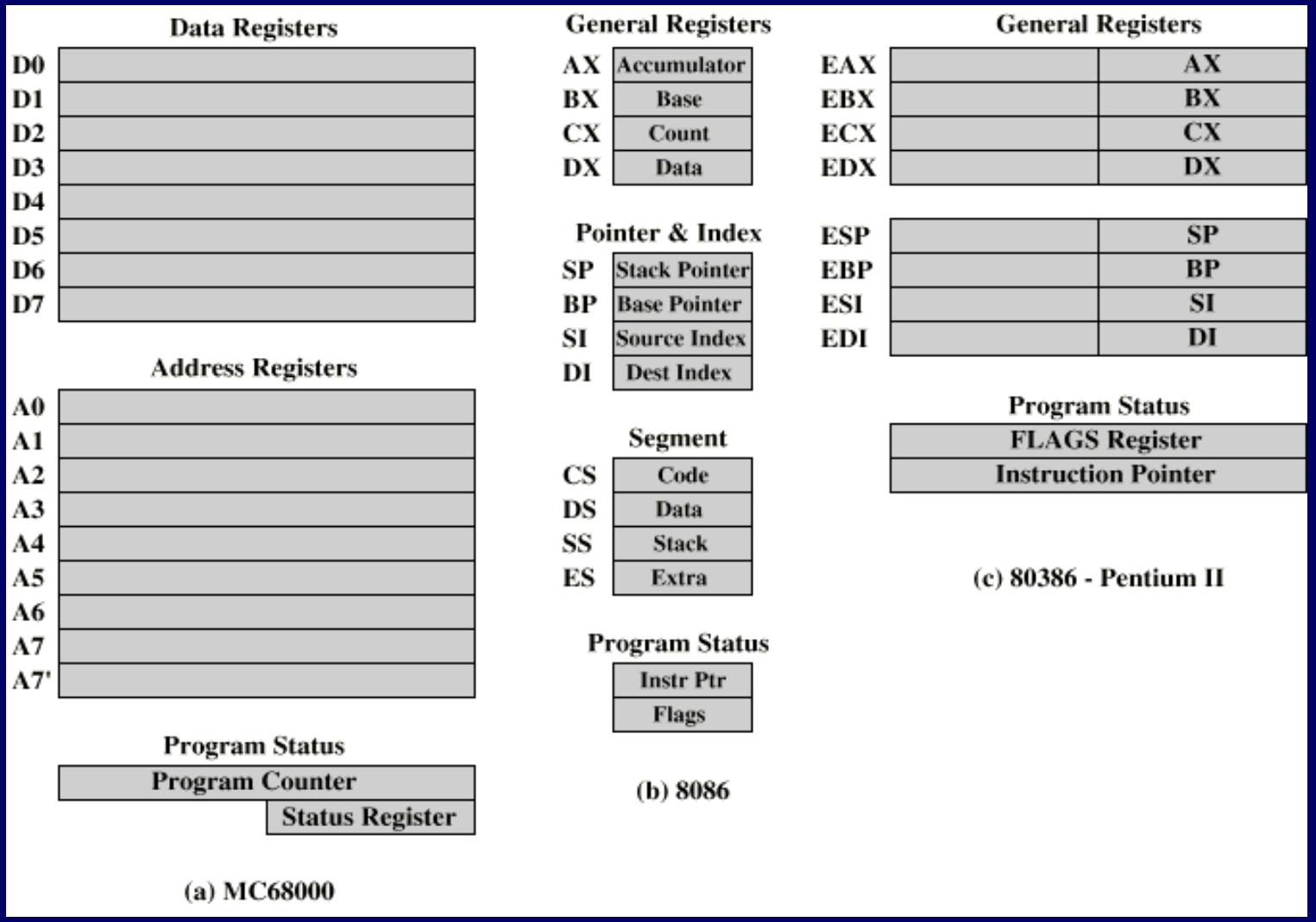
Intel 8086 Register Organisation (2)

- 4 16-bit segment registers
  - Code, Data, Stack & 'extra' registers
  - used to point to segment of current instruction (branch), segment with data, or segment with stack
- 1 instruction pointer
- A set of 1-bit status & control flags

Register Organisation
The point is...

- No best way to organise CPU registers
- For upward compatibility, the later Pentium II (32-bit) retains original 8086 register organisation
  - 8086 register organisation embedded in new organisation

Example Register Organizations



Outline

- Processor & Register Organisation
  - CPU structure
  - User-visible registers
  - Control & status registers
- Instruction Cycle
  - Fetch, execute, interrupt & indirect cycle
  - Data flow
- Instruction Pipelining
  - Performance
  - Branch

Instruction Cycle

- Instruction cycle includes the following subcycles:
  - Fetch - read next instruction from memory into CPU
  - Execute - interpret opcode and perform operation
  - Interrupt - save current process state and service interrupt
  - Indirect – more memory access to fetch operand

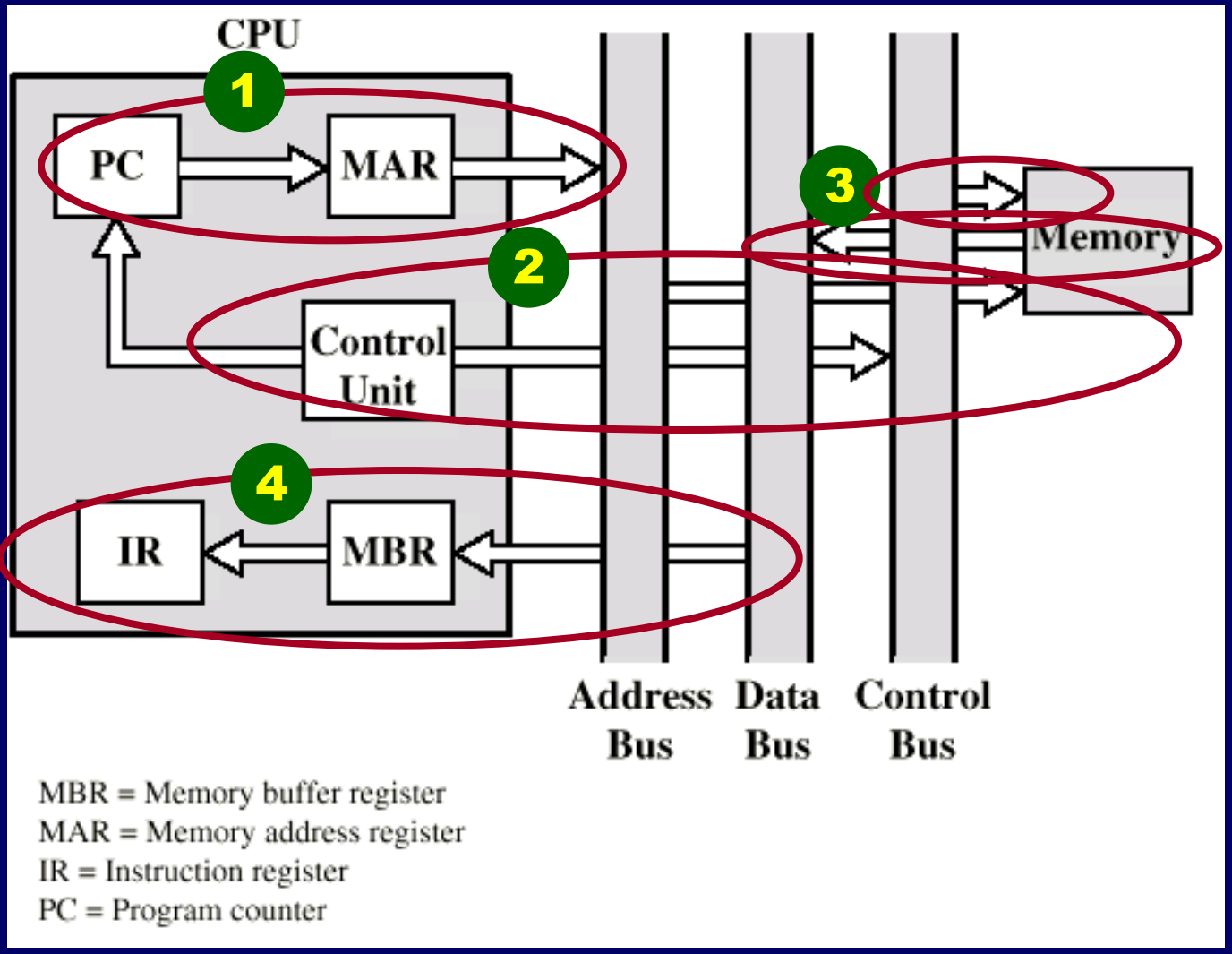
Fetch Cycle

- Program Counter (PC) holds address of next instruction to fetch
- Processor fetches instruction from memory location pointed to by PC
- Increment PC
  - Unless told otherwise
- Instruction loaded into Instruction Register (IR)
- Processor interprets instruction and performs required actions

Data Flow (Instruction Fetch)

- Data flow depends on CPU design
- For fetch cycle (instruction read from memory):
  - PC contains address of next instruction
  - Address moved to MAR
  - Address placed on address bus
  - Control unit requests memory read
  - Result placed on data bus, copied to MBR, then to IR
  - Meanwhile PC incremented by 1

Data Flow (Fetch Diagram)



Execute Cycle

- Processor-memory
  - data transfer between CPU and main memory
- Processor I/O
  - Data transfer between CPU and I/O module
- Data processing
  - Some arithmetic or logical operation on data
- Control
  - Alteration of sequence of operations
  - e.g. jump
- Combination of above

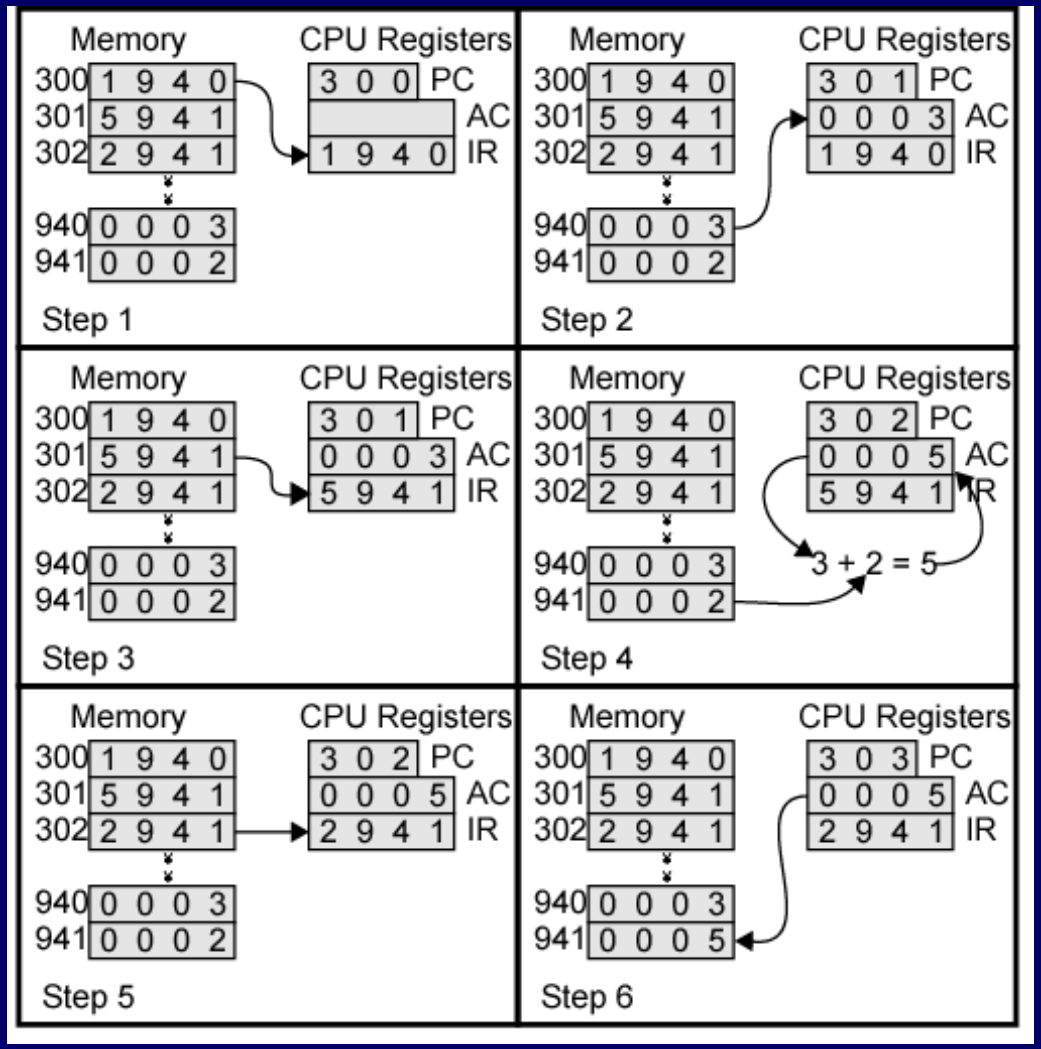
Data Flow (Execute)

- May take many forms
- Depends on instruction (in IR) being executed
- May include
  - Memory read/write
  - Input/Output
  - Register transfers
  - ALU operations

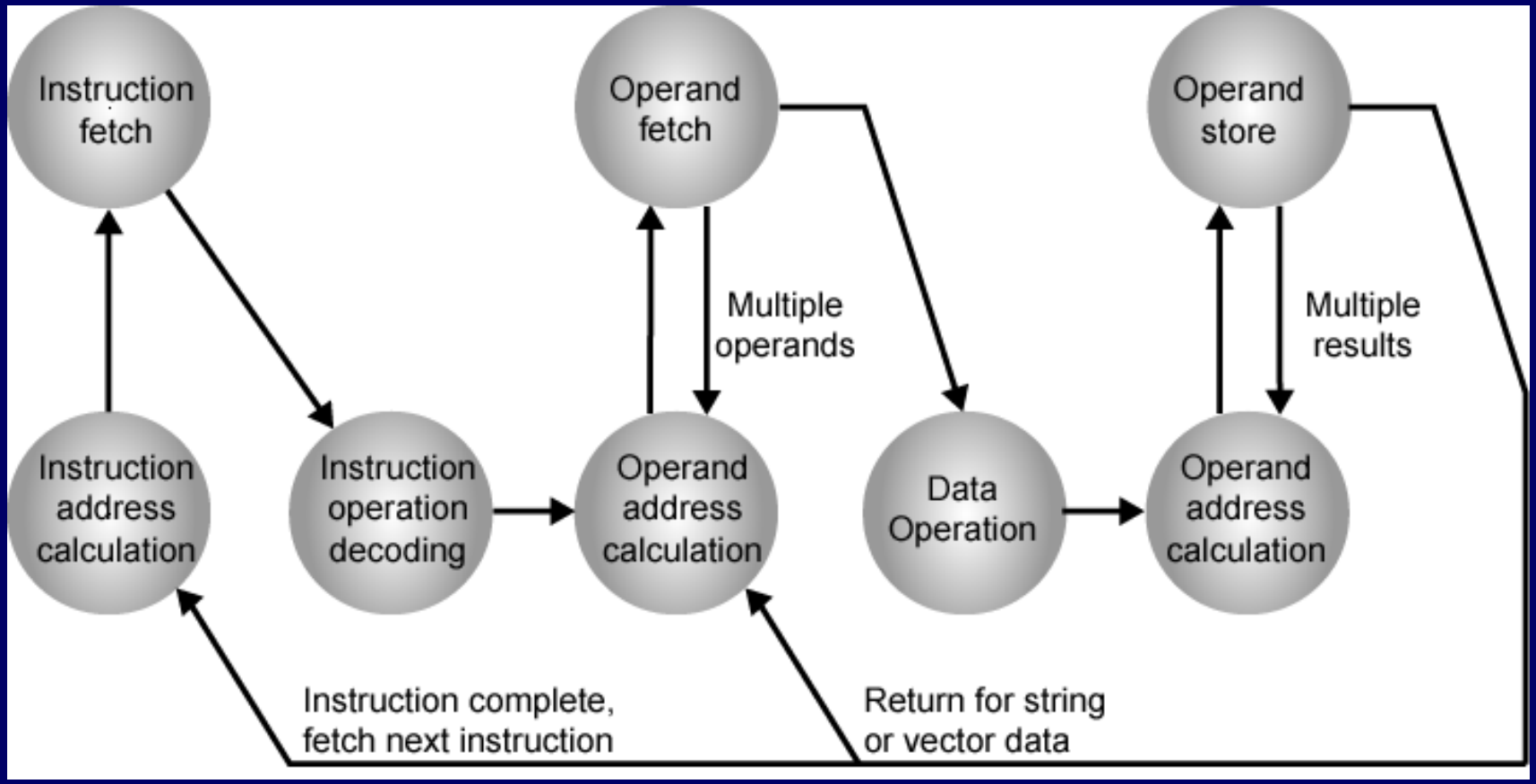
Example of Program Execution

Opcode	Address
1	3

1 = Load
2 = Store
5 = Add



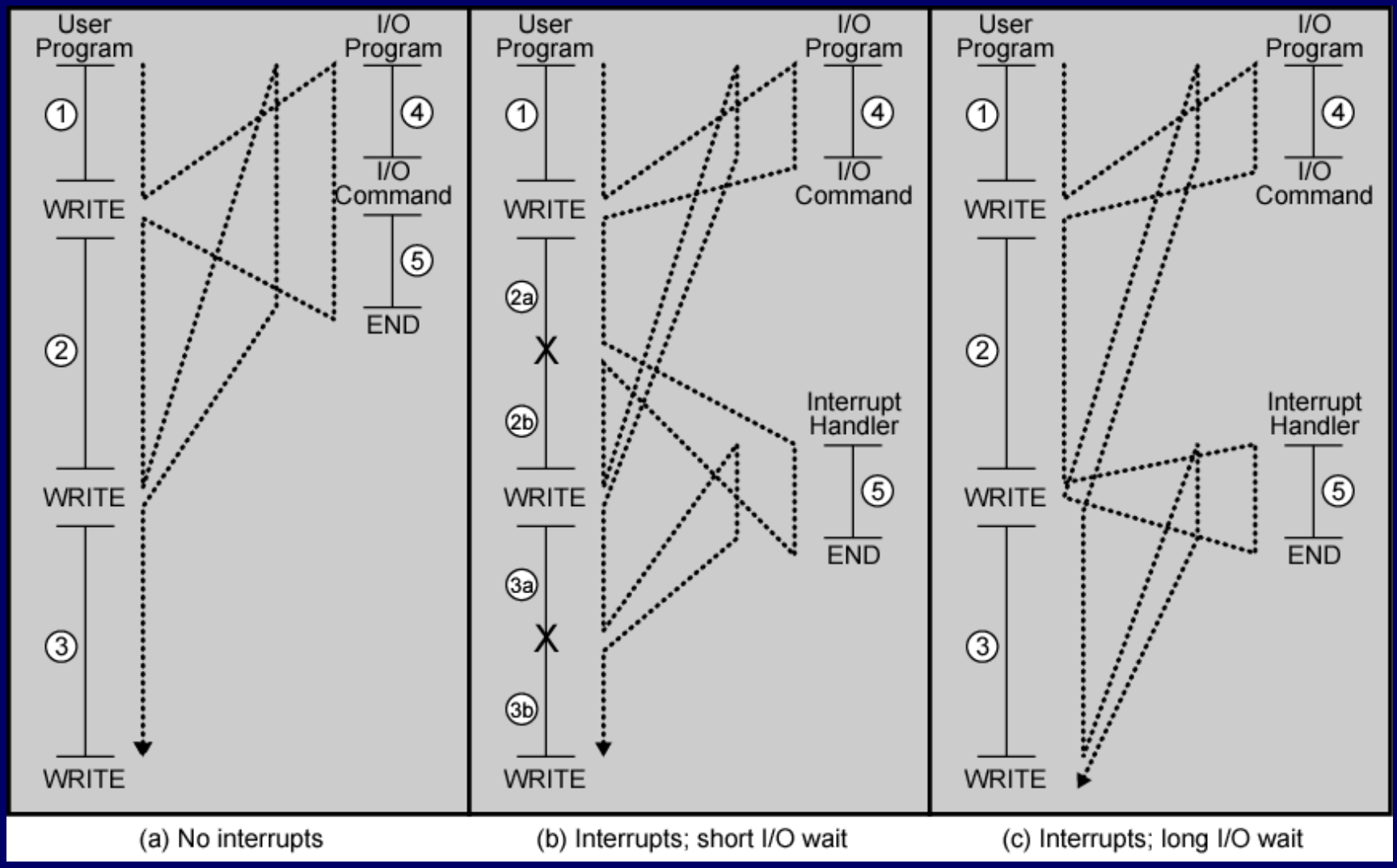
Instruction Cycle State Diagram



Interrupts

- Mechanism by which other modules (e.g. I/O) may interrupt normal sequence of processing
- Program
  - e.g. overflow, division by zero
- Timer
  - Generated by internal processor timer
  - Used in pre-emptive multi-tasking
- I/O
  - from I/O controller
- Hardware failure
  - e.g. memory parity error

Program Flow Control



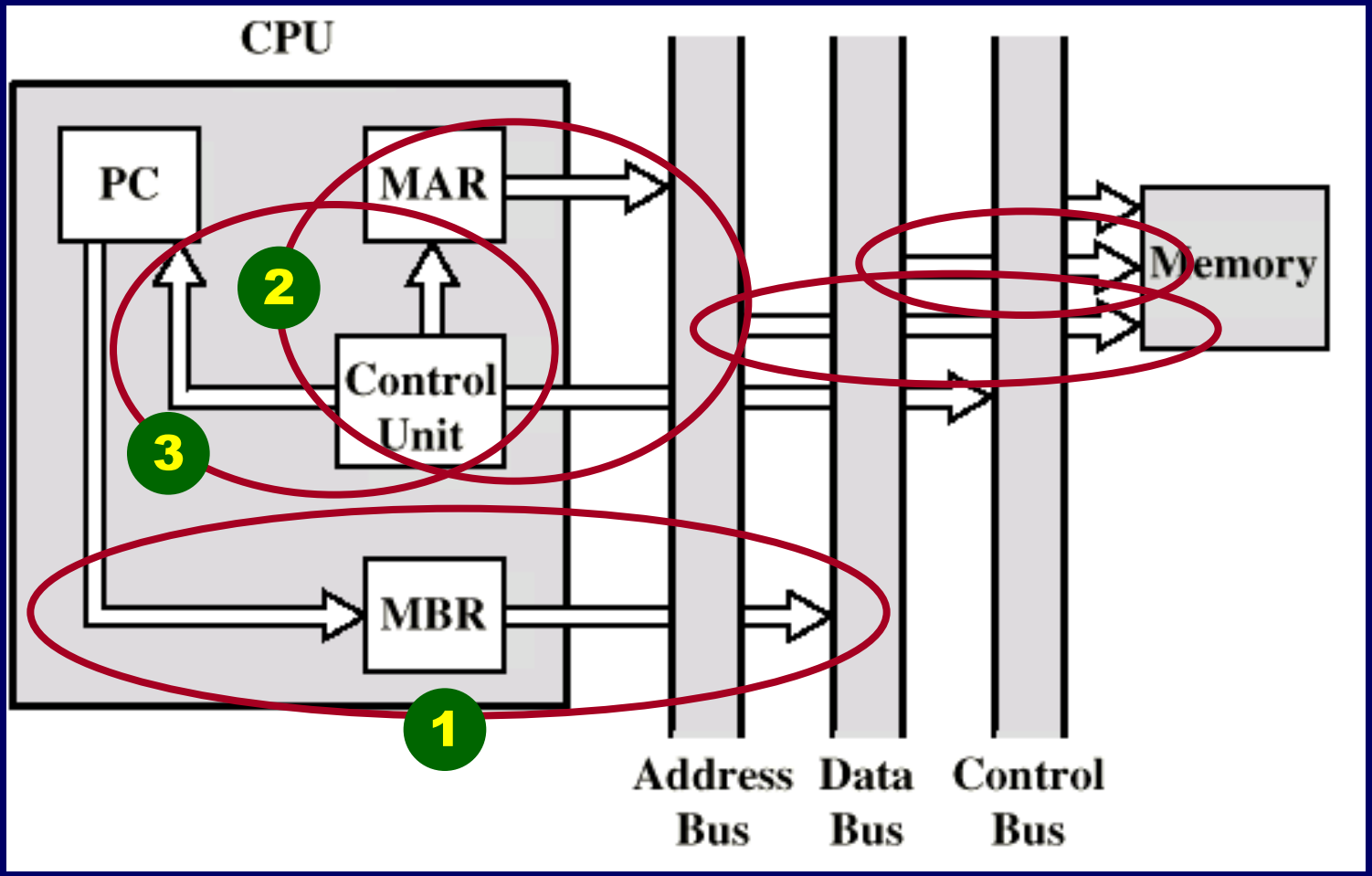
Interrupt Cycle

- Added to instruction cycle
- Processor checks for interrupt
  - Indicated by an interrupt signal
- If no interrupt, fetch next instruction
- If interrupt pending:
  - Suspend execution of current program
  - Save context
  - Set PC to start address of interrupt handler routine
  - Process interrupt
  - Restore context and continue interrupted program

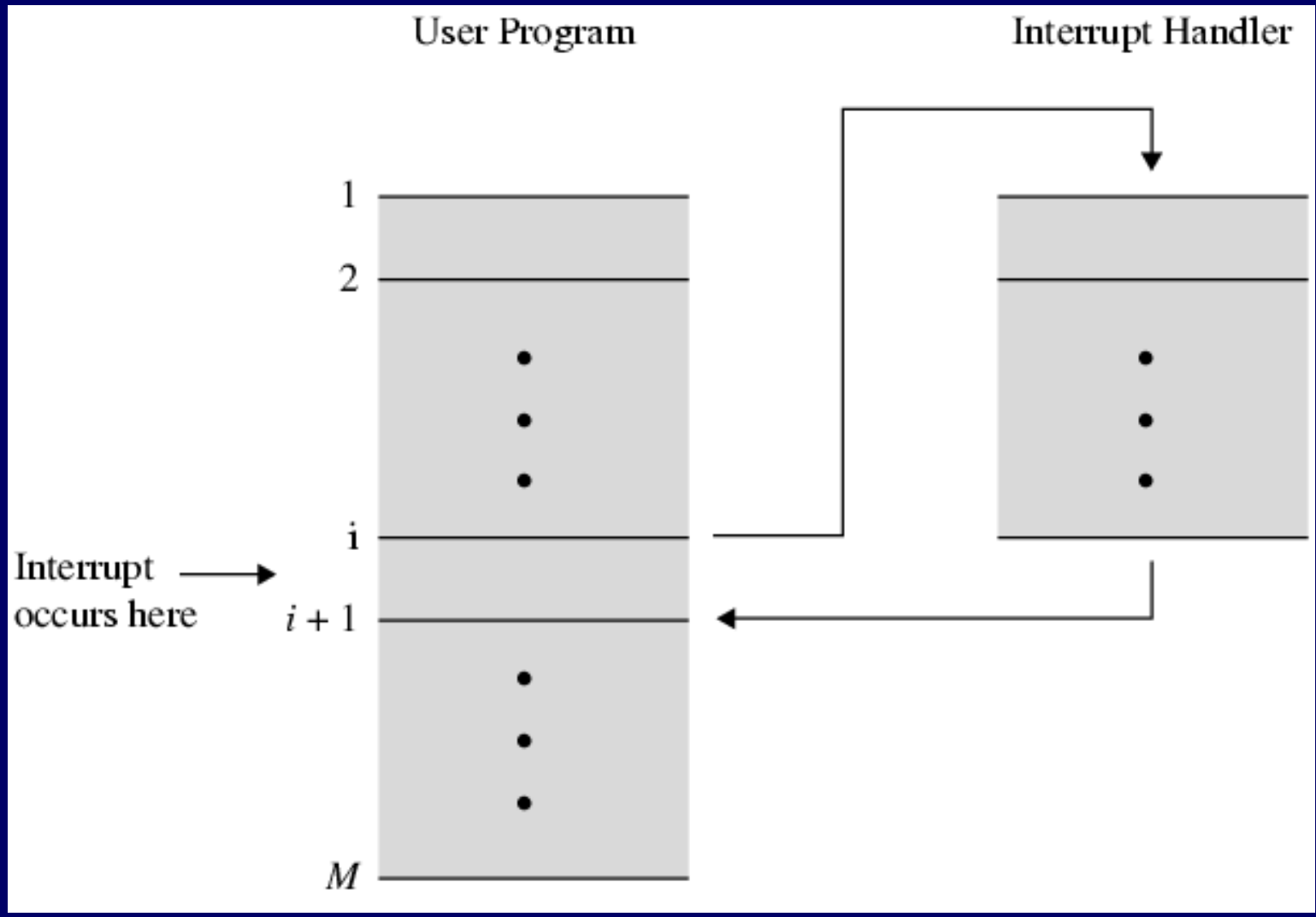
Data Flow (Interrupt)

- Simple
- Predictable
- Current PC saved to allow resumption after interrupt
  - Contents of PC copied to MBR
  - Then, MBR written to memory
  - Special memory location (e.g. stack pointer) loaded to MAR
- PC loaded with address of interrupt handling routine
- Next instruction (first of interrupt handler) can be fetched

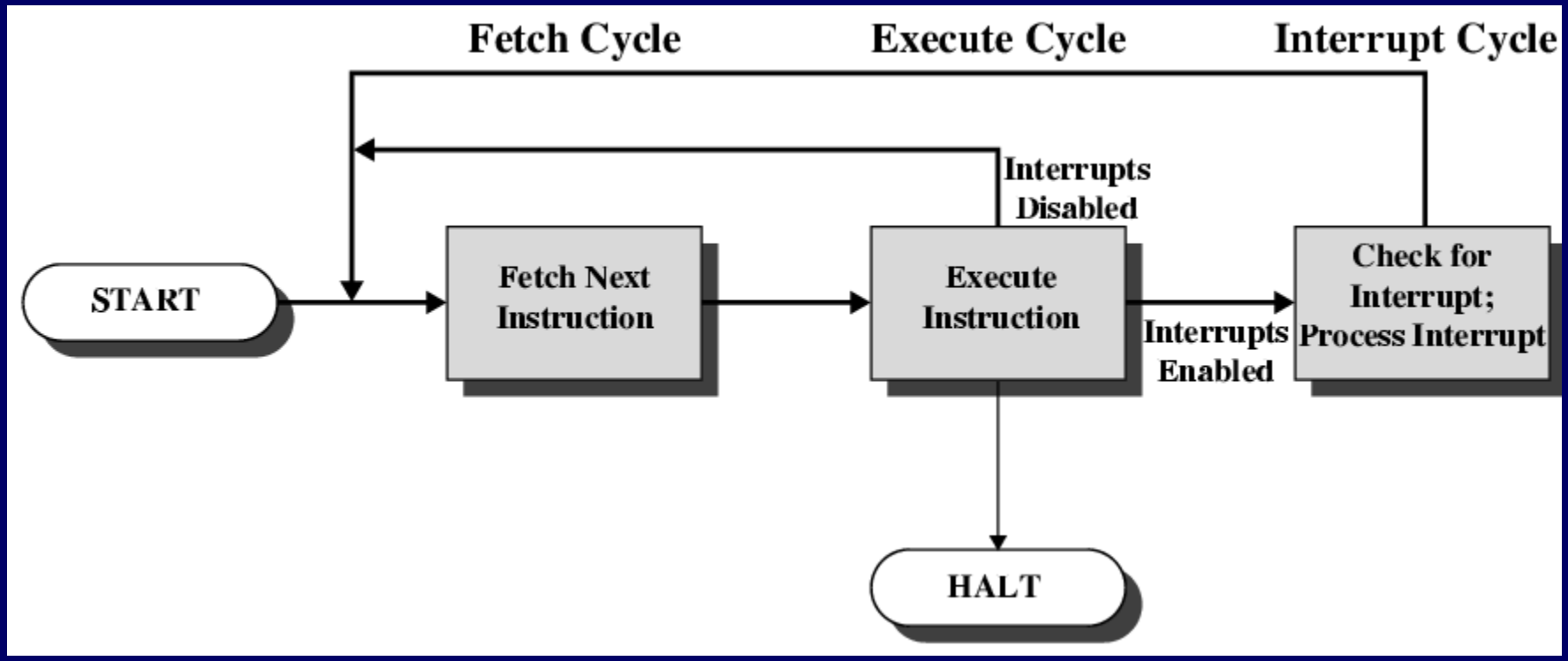
Data Flow (Interrupt Diagram)



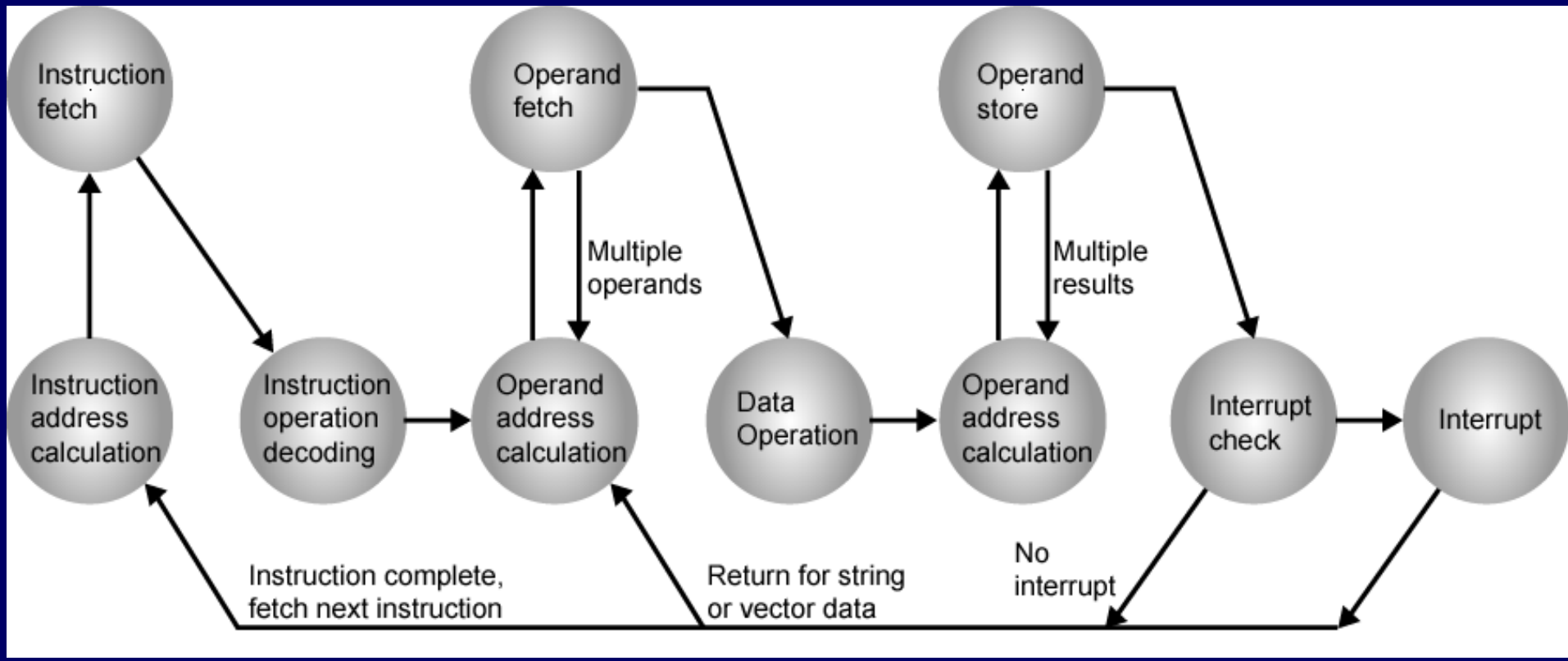
Transfer of Control via Interrupts



Instruction Cycle with Interrupts



Instruction Cycle (with Interrupts) - State Diagram



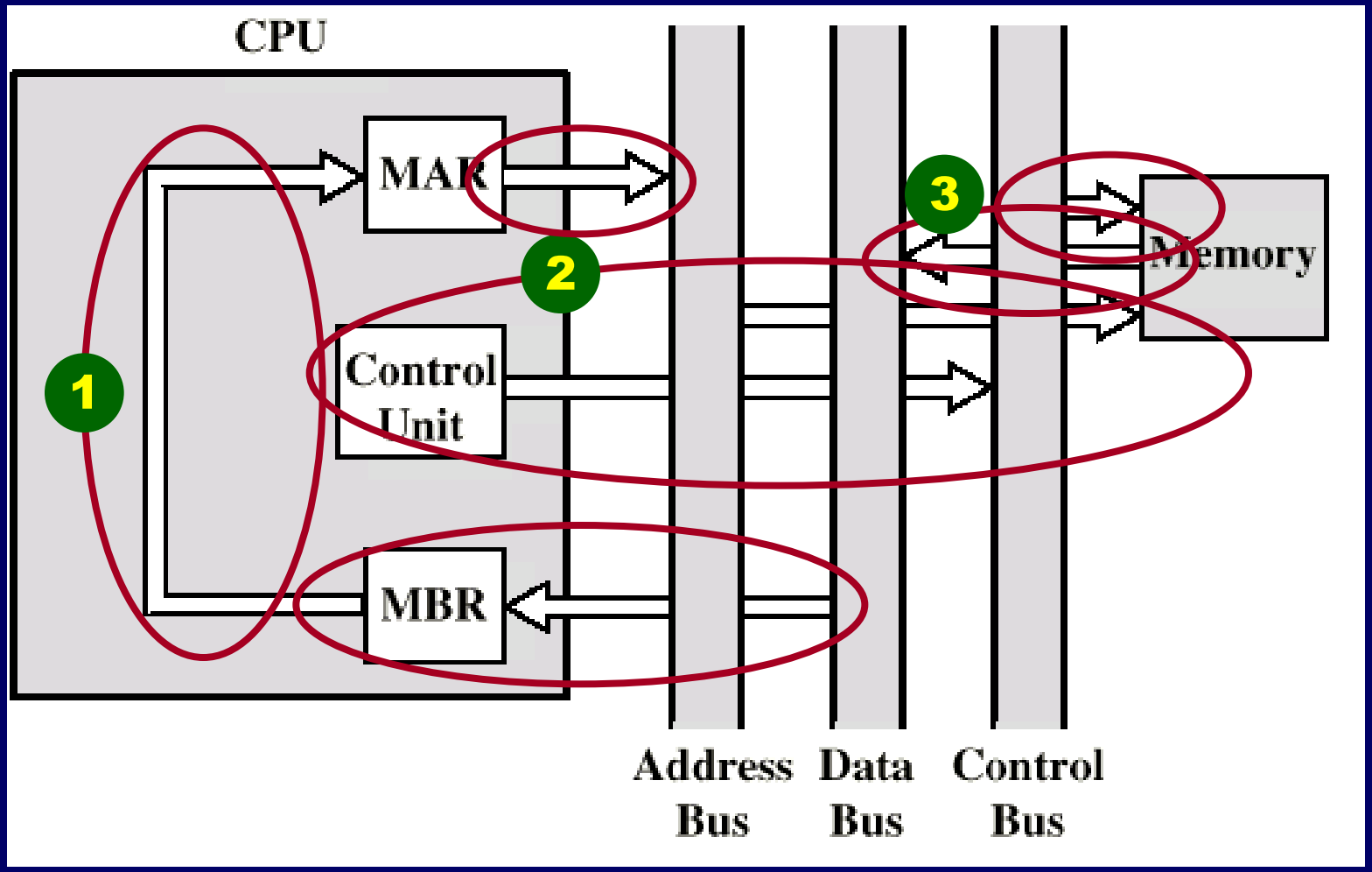
Indirect Cycle

- May require memory access to fetch operands
- Indirect addressing requires more memory accesses
- Can be thought of as additional instruction subcycle

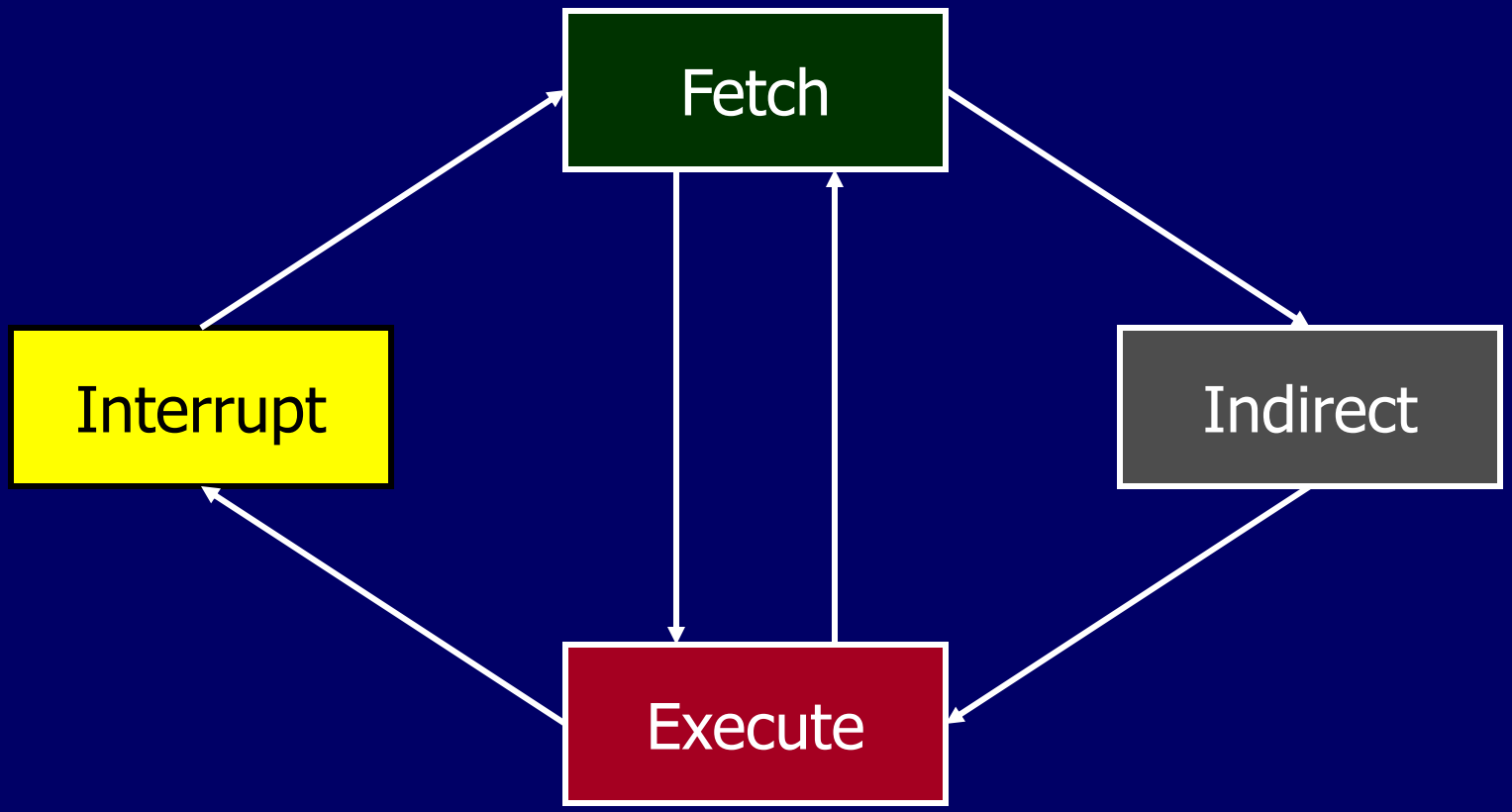
Data Flow (Indirect)

- After fetch, IR is examined
- If indirect addressing, indirect cycle is performed
  - Right most N bits of MBR transferred to MAR
  - Control unit requests memory read
  - Result (address of operand) moved to MBR

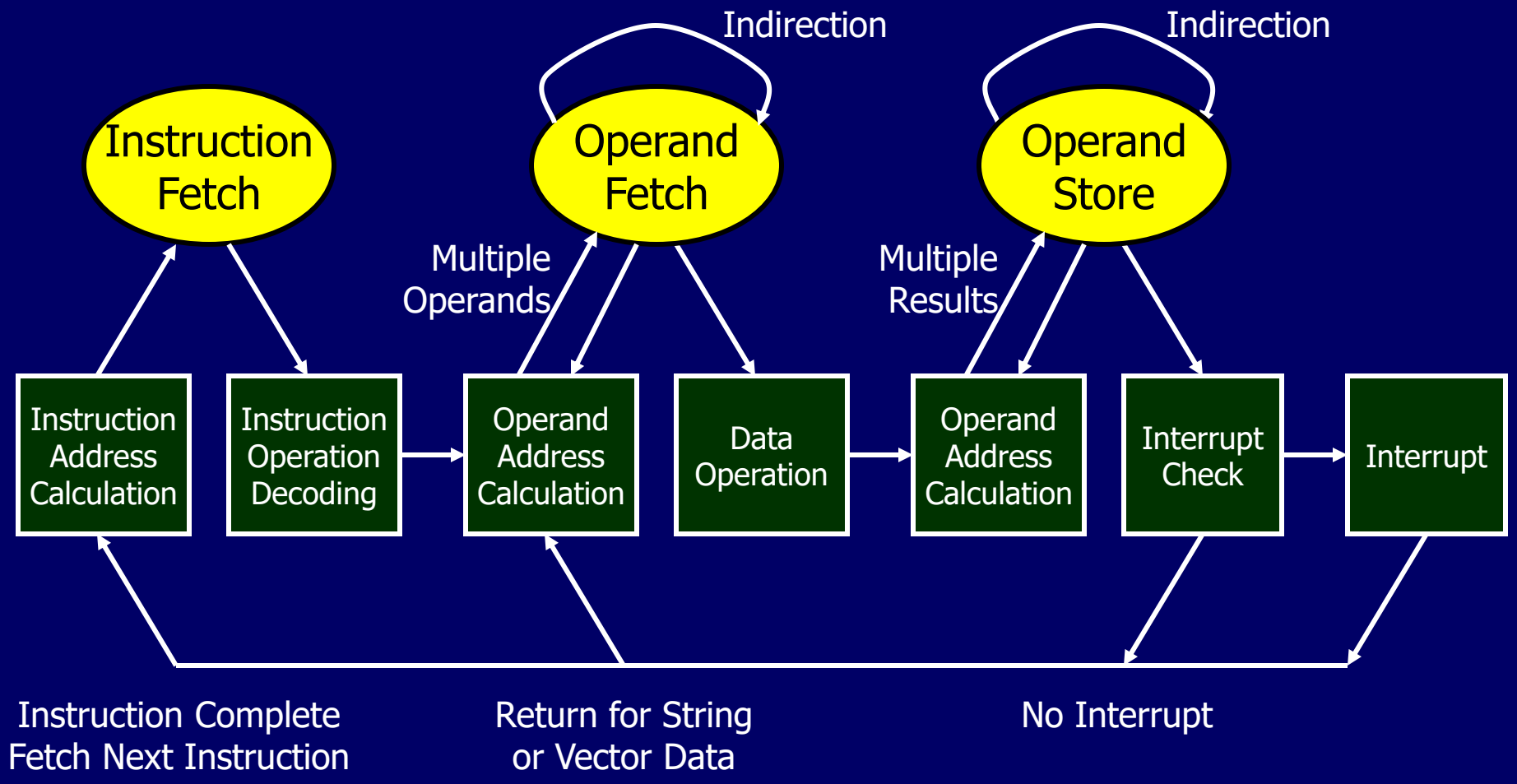
Data Flow (Indirect Diagram)



Instruction Cycle with Indirect



Instruction Cycle State Diagram



Outline

- Processor & Register Organisation
  - CPU structure
  - User-visible registers
  - Control & status registers
- Instruction Cycle
  - Fetch, execute, interrupt & indirect cycle
  - Data flow
- Instruction Pipelining
  - Performance
  - Branch

Instruction Pipelining Prefetch

- Analogy: assembly line
  - Various stages of product can be worked on simultaneously = pipelining
- For computers:
  - Execution usually does not access main memory
  - Fetch accessing main memory
- Can fetch next instruction during execution of current instruction (simple 2-stage pipeline)
- Called instruction prefetch

Improved Performance

- If fetch and execute is equal duration, instruction cycle time will be halved
- But (speed) not doubled:
  - Fetch usually shorter than execution
    - execution needs reading & storing operands, etc
    - fetch stage will need to wait before it can pass on to execute
  - Any jump or branch means that prefetched instructions are not the required instructions
    - time loss can be reduced by guessing (e.g. always guess next instruction)
- Add more stages to improve performance

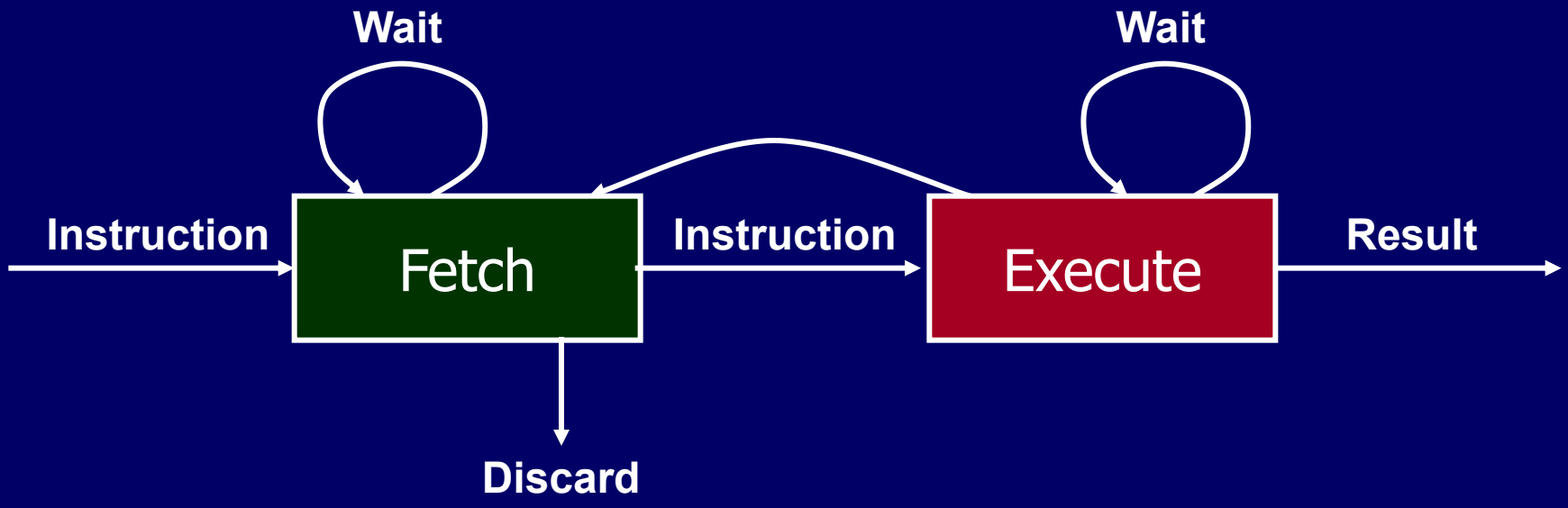
Pipelining

- To gain further speedup, pipeline must have more stages:
  - Fetch instruction: read next instruction
  - Decode instruction: determine opcode & operand specifiers
  - Calculate operands (i.e. EAs): addressing mode
  - Fetch operands: from memory (for register, no need)
  - Execute instructions: execute and store results in dest
  - Write result: in memory
- More decomposition, more equal stage duration
- Overlap these operations

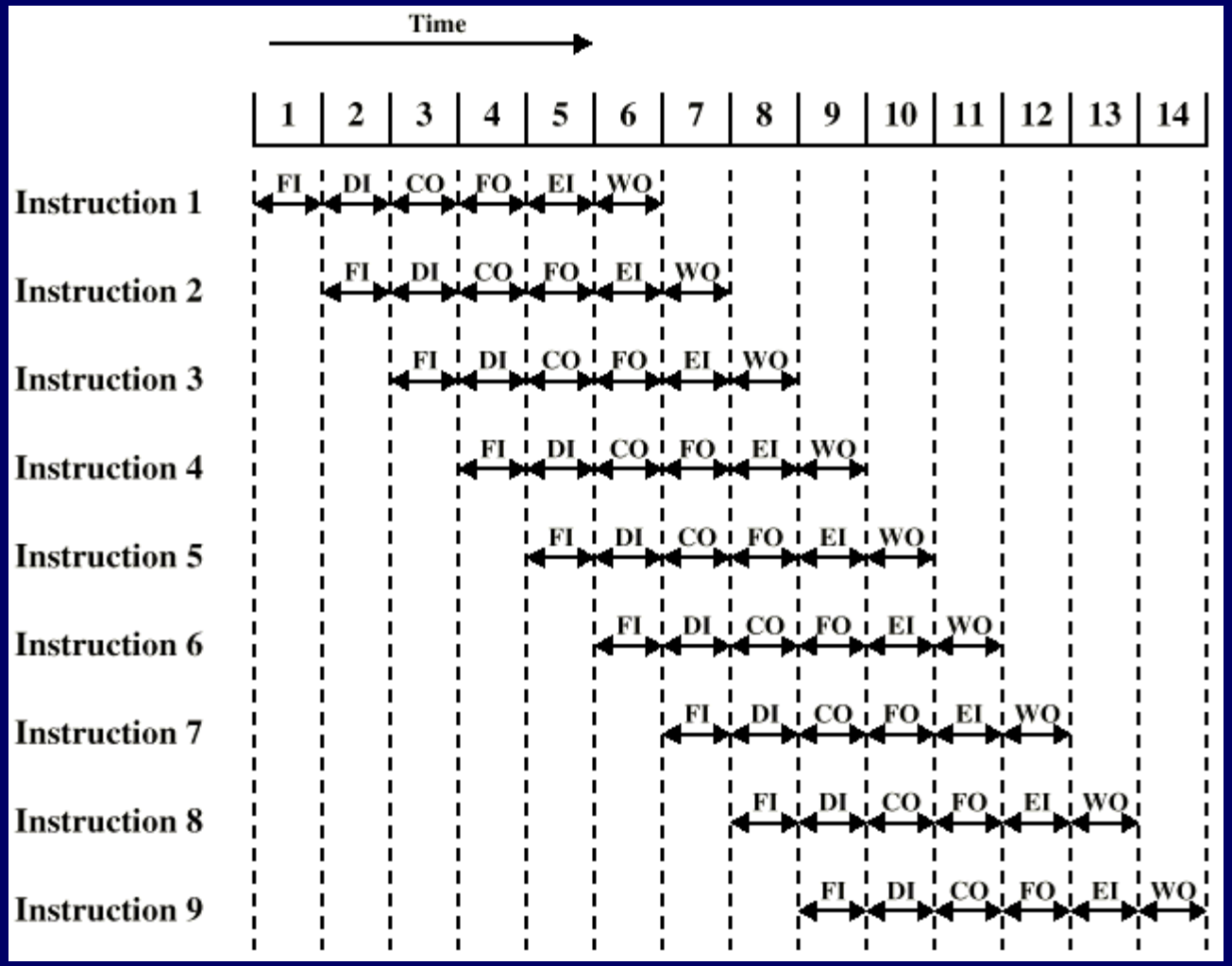
Two Stage Instruction Pipeline

Simplified View

Expanded View



Timing of Pipeline (1)



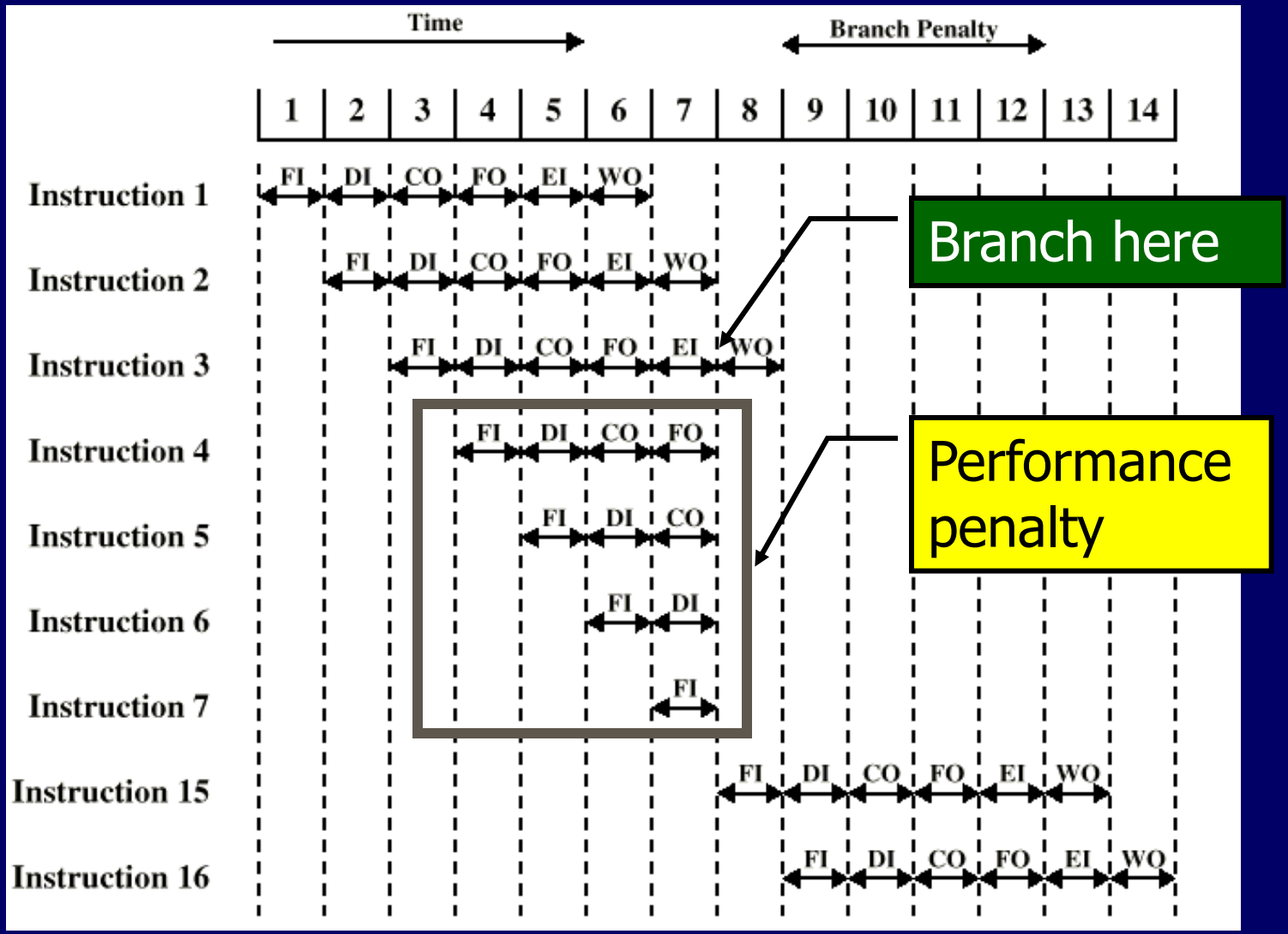
Timing of Pipeline (2)

- Assumptions
  - each instruction require all 6 stages
    - e.g. LOAD instruction does not need WO stage
  - all stages can execute in parallel
    - assumes no memory conflicts
    - but e.g. FO, EI & WO accesses memory (usually not allowed simultaneous access)

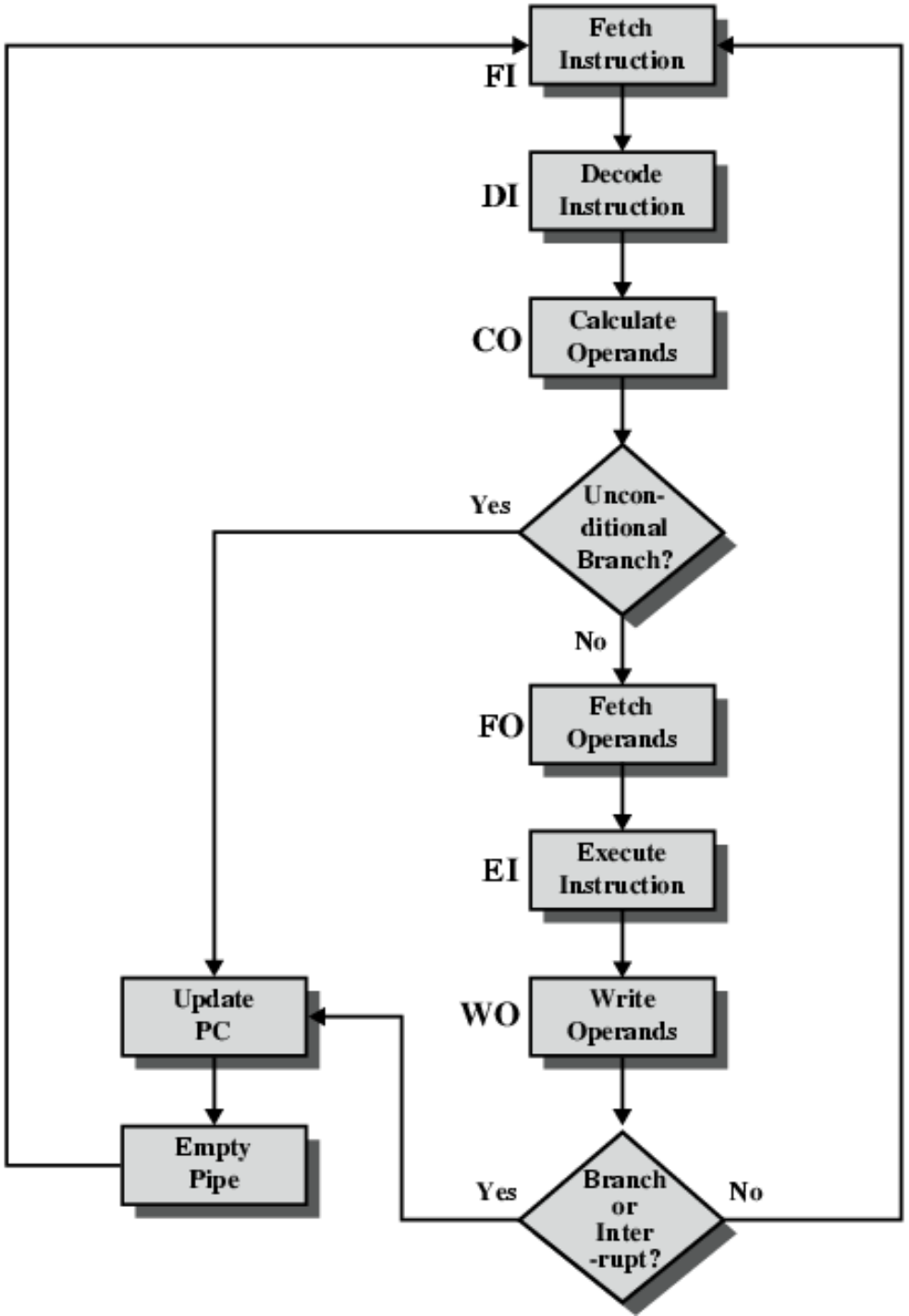
Pipelining Performance Limits

- Waiting involved if stages are not of equal duration (like in 2-stage pipeline)
- Conditional branches
- Interrupts
- Calculate Operand stage may see conflict in memory and register
- Increasing number of stages not necessary improve performance:
  - Overhead of moving data from buffer to buffer & performing preparatory and delivery functions
  - More control logic required- more complex than stages

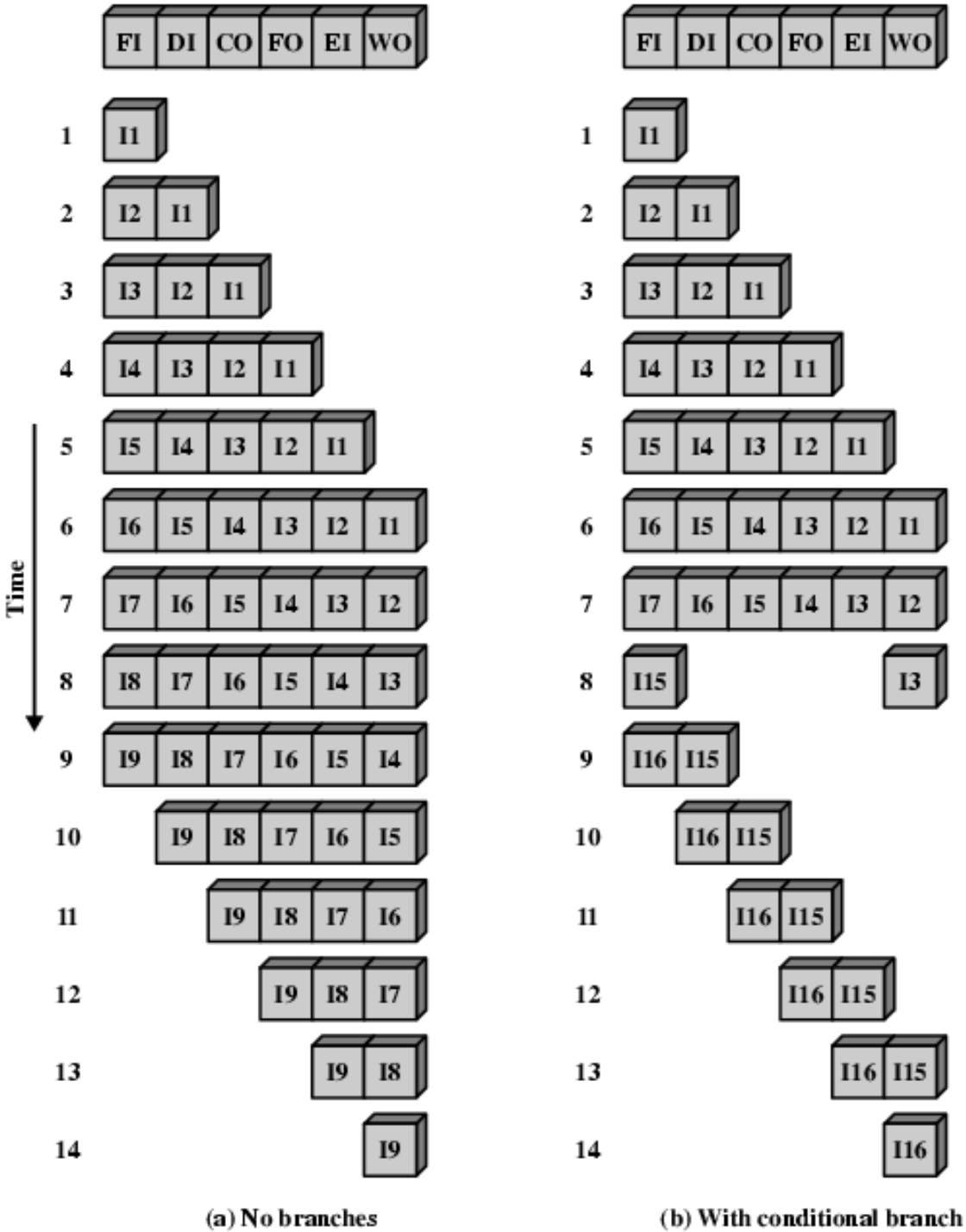
Branch in a Pipeline



Six Stage Instruction Pipeline (Branch logic)



Alternative Pipeline Depiction



Pipeline Performance (1)

- Without pipeline,

  $T_1 = nk$, where

  T = Execution time

  n = number of iterations (instructions)

  k = number of stages

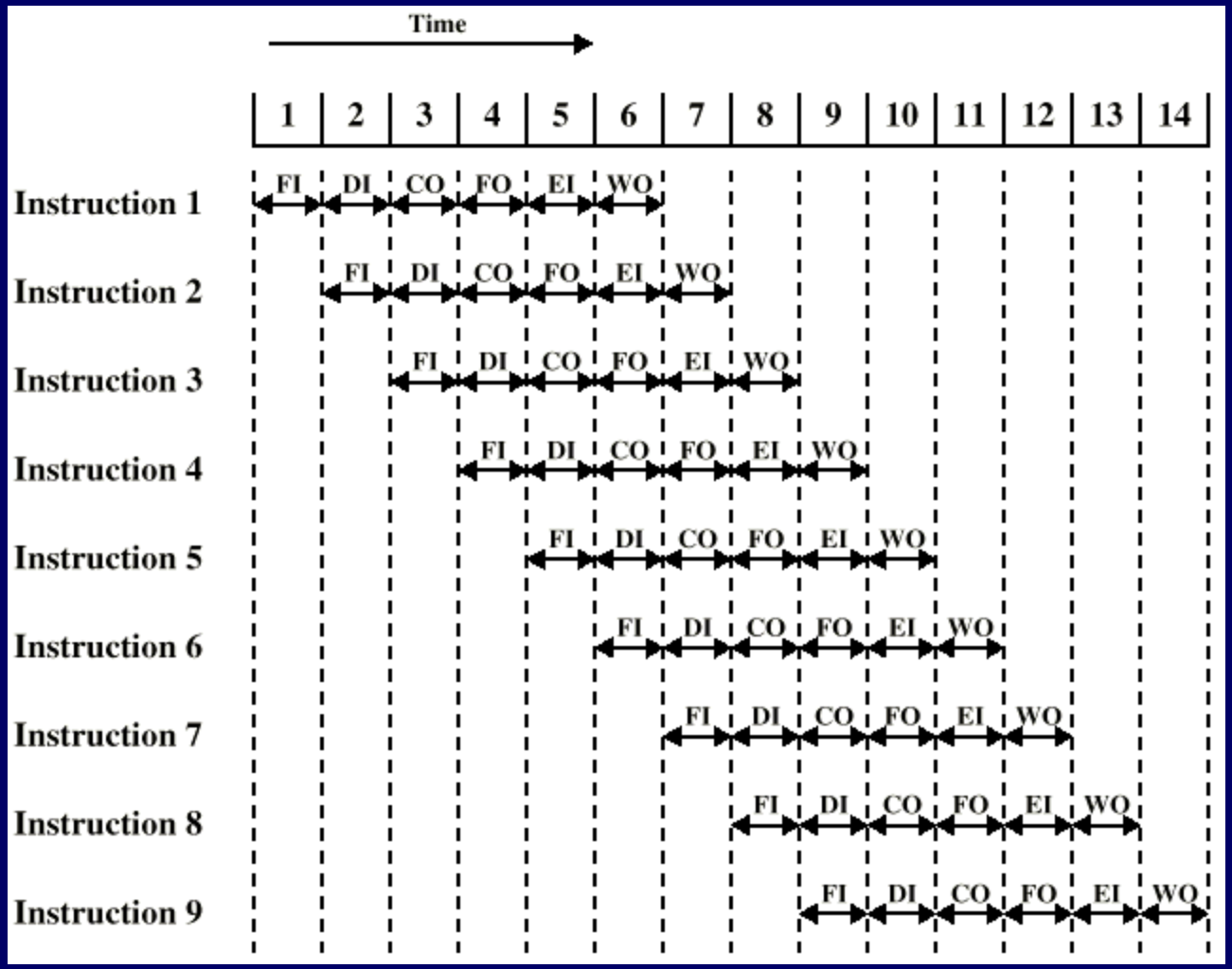
- With pipeline,

  $T_k = k + (n - 1)$

- Speedup,

  $S = T_1 / T_k = nk / k + (n - 1)$

Pipeline Performance (2)



Without pipeline:

$T_1 = nk$

$= 9 \times 6$

$= 54$

With pipeline:

$T_k = k + (n-1)$

$= 6 + (9-1)$

$= 6 + 8$

$= 14$

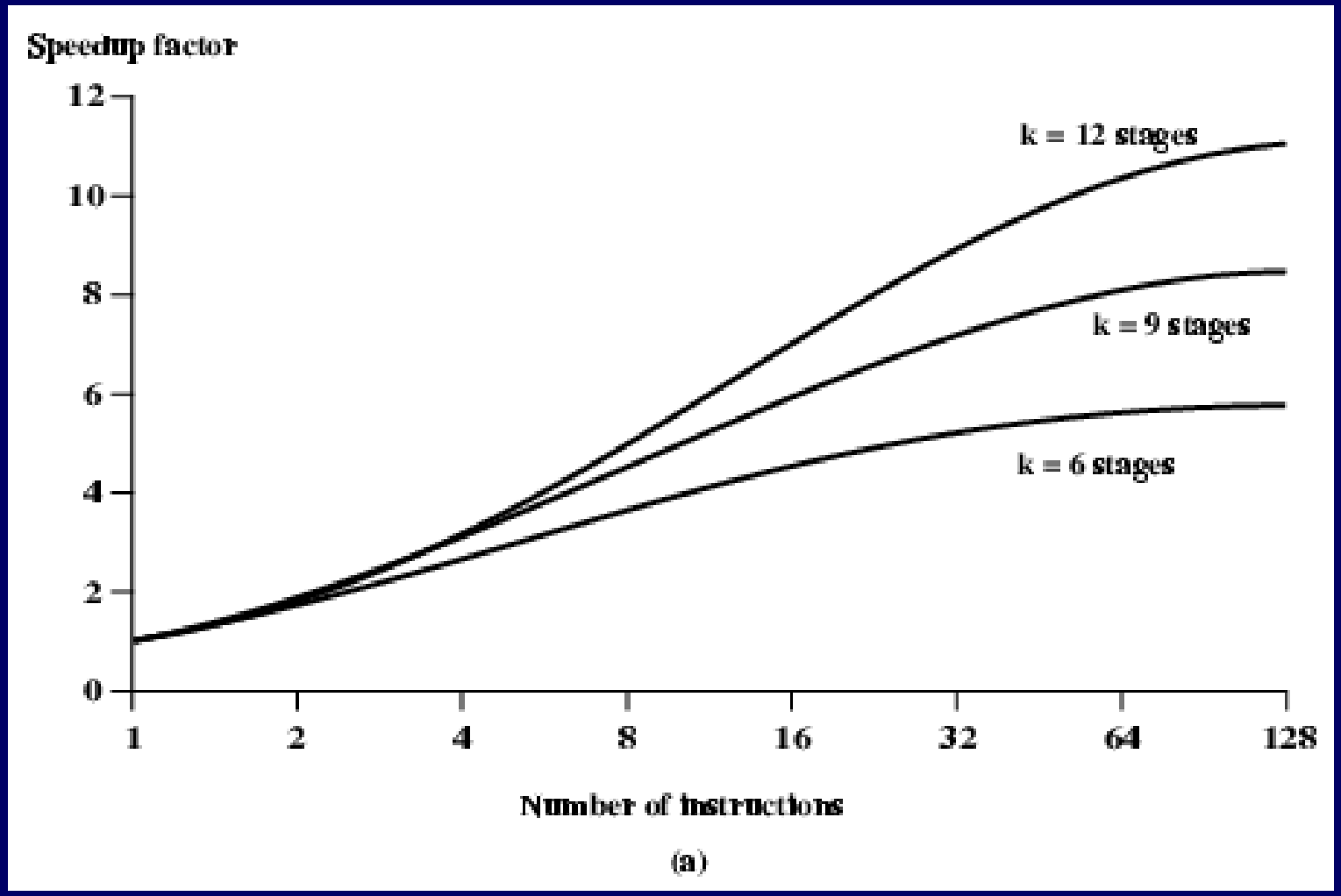
Speedup:

$S = T_1 / T_k$

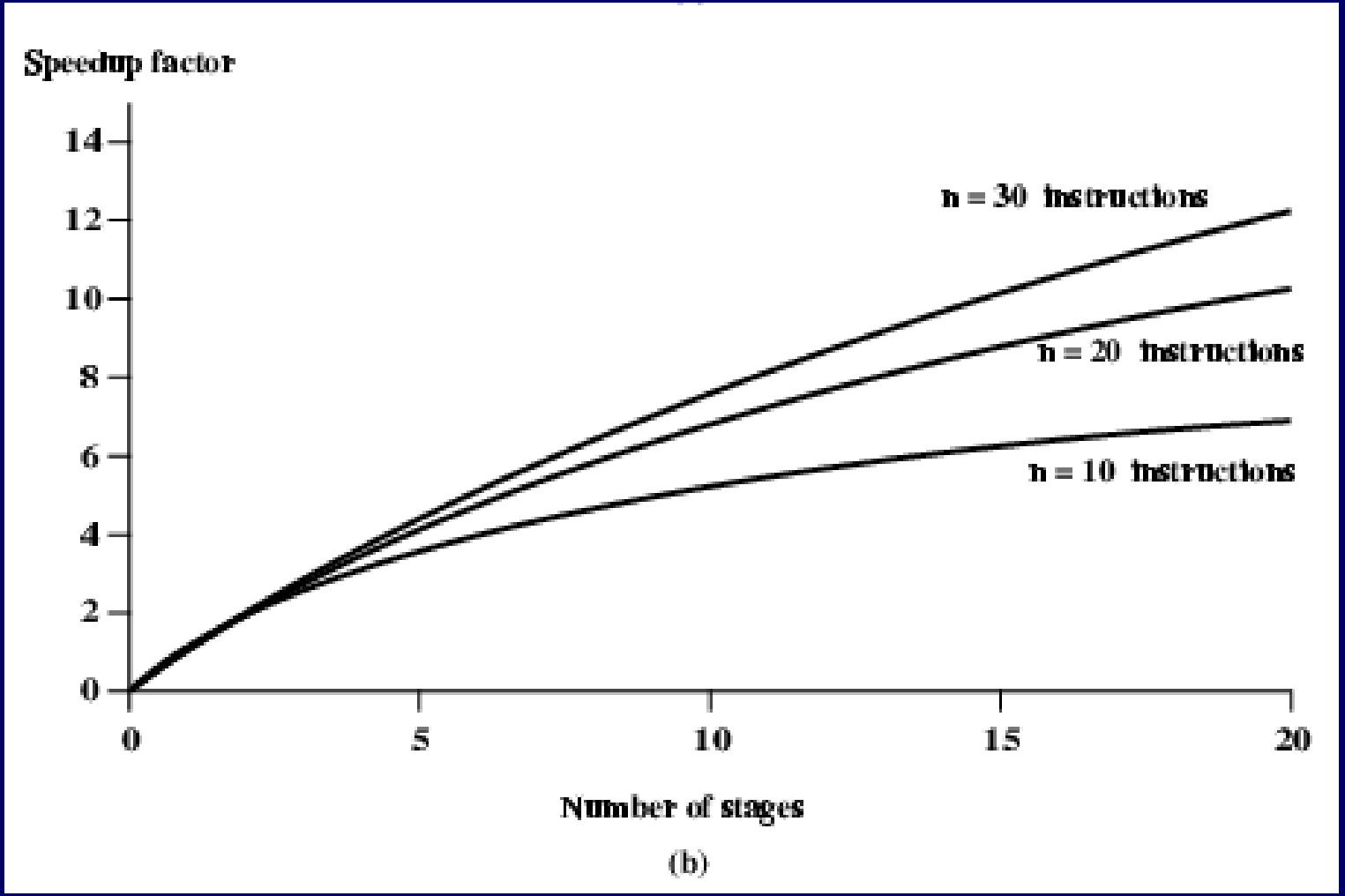
$= 54 / 14$

$= 3.86$

Speedup Factors with Instruction Pipelining (1)



Speedup Factors with Instruction Pipelining (2)



Dealing with Branches

- Conditional branches are a problem in pipelines
- Approaches to deal with conditional branches:
  - Multiple Streams
  - Prefetch Branch Target
  - Loop buffer
  - Branch prediction
  - Delayed branching

Multiple Streams

- Simple pipeline suffers penalty because need to choose 1 of 2 instructions to fetch
- Solution: have two pipelines
- Prefetch each branch into a separate pipeline
- Then use appropriate pipeline
- Problems:
  - Leads to bus & register contention delays
  - Multiple branches lead to further pipelines being needed (branch in branch)
- e.g. in IBM 370/168 & IBM 3033

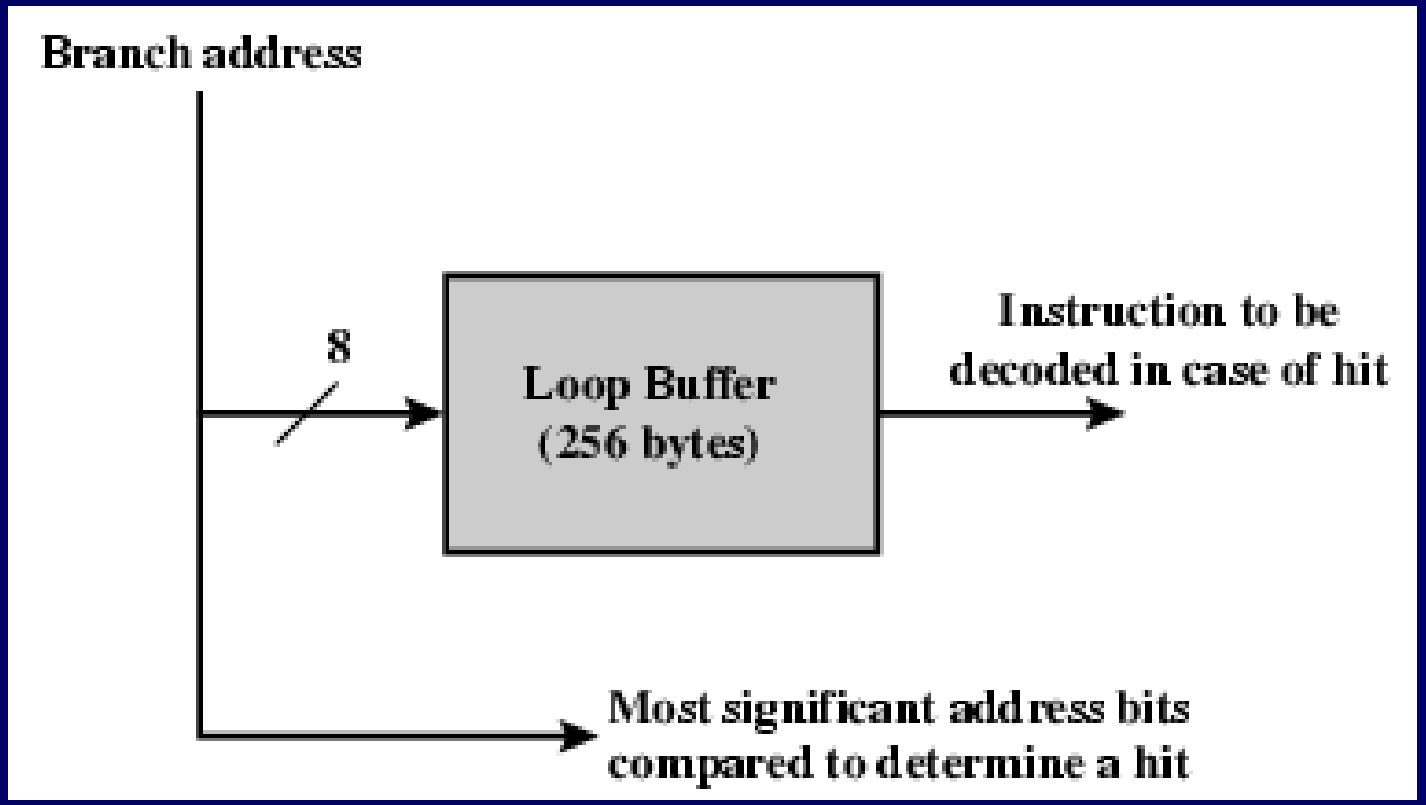
Prefetch Branch Target

- Target of branch is prefetched in addition to instructions following branch
- Keep target until branch is executed
- e.g. in IBM 360/91

Loop (Look Ahead Look Behind) Buffer

- Very fast memory
- Maintained by fetch stage of pipeline
- Buffer has *n* most recently fetched instructions
- Check buffer before fetching from memory
- Benefits:
  - buffer may contain some instructions ahead
  - branch target may be just a few locations away
  - very good for small loops or jumps
- Similar to cache (but sequential instructions)
- Used by CRAY-1

Loop Buffer Diagram



Branch Prediction (1)

- Guess next instruction to execute
- If guessed right, no loss of performance
- If guessed wrong, empty pipeline and restart
- Static guesses
  - Branch never taken
  - Branch always taken
  - Predict based on opcode
- Dynamic guesses
  - Taken/not taken switch
  - History table

Branch Prediction (2)

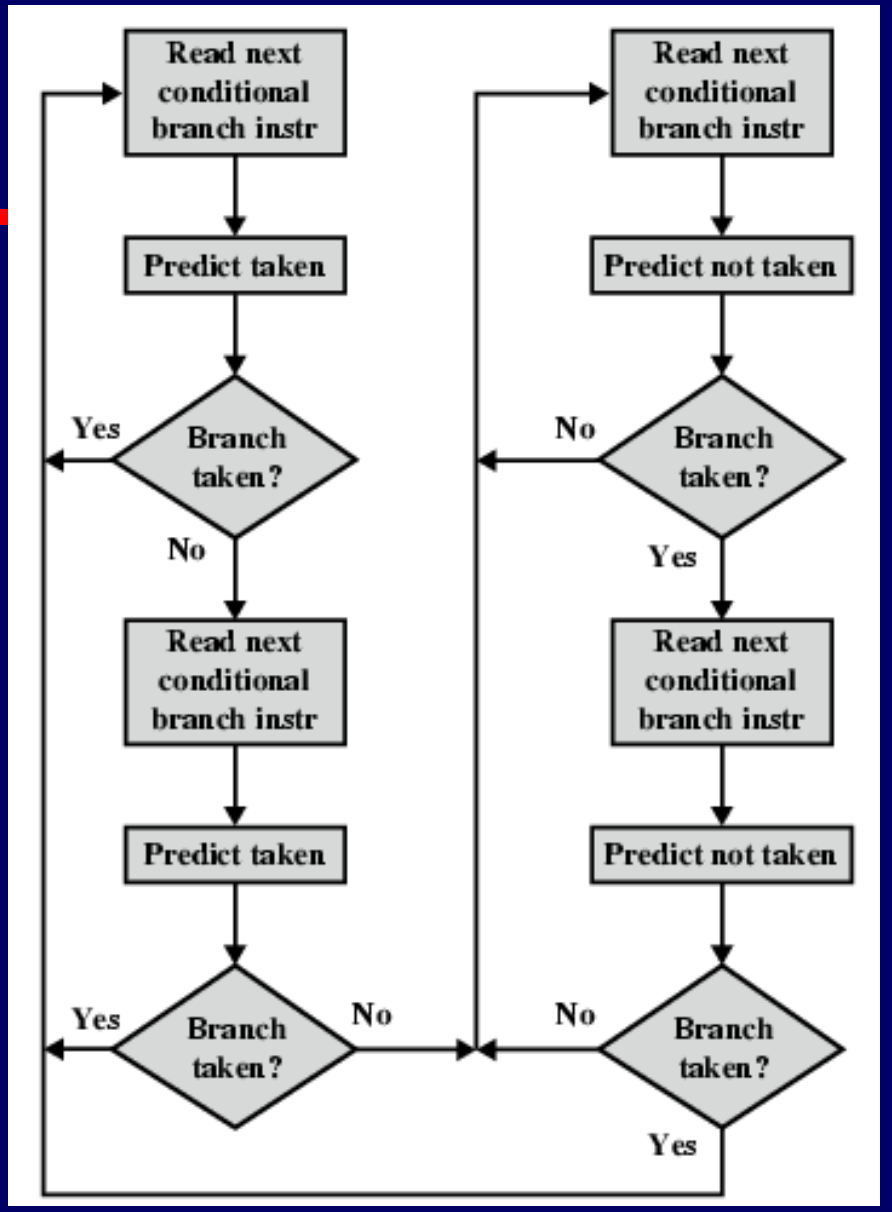
- Predict never taken
  - Assume that jump will not happen
  - Always fetch next instruction
  - 68020 & VAX 11/780
  - VAX will not prefetch after branch if a page fault would result (O/S v CPU design)
- Predict always taken
  - Assume that jump will happen
  - Always fetch target instruction
  - More than 50% of conditional branches are taken

Branch Prediction (3)

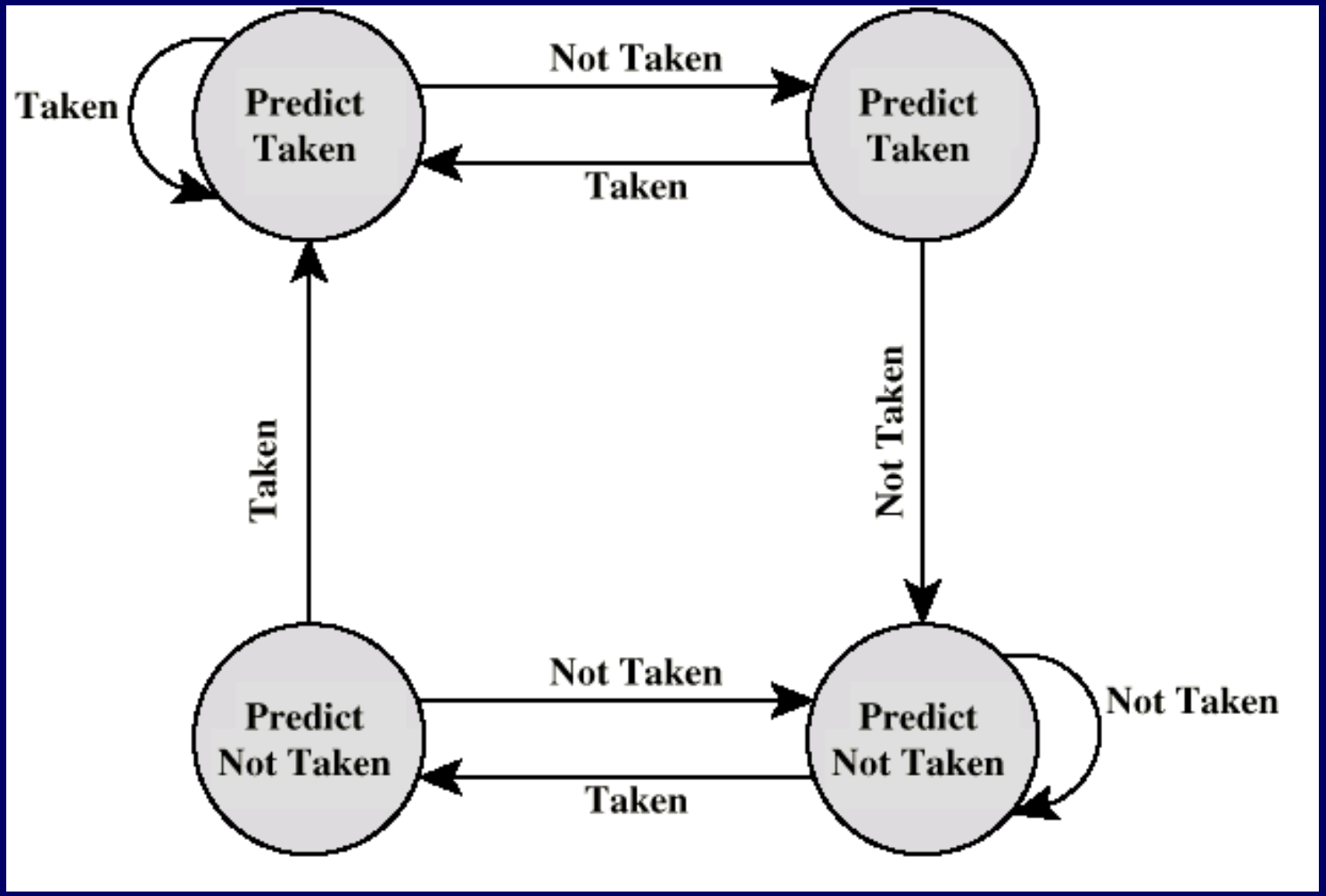
- Predict by Opcode
  - Some instructions are more likely to result in a jump than others
  - Can get up to 75% success
- Taken/Not taken switch
  - Based on previous history
  - Good for loops
  - e.g. in IBM 3090/400

Branch Prediction (4)

Taken/ Not Taken: Flowchart



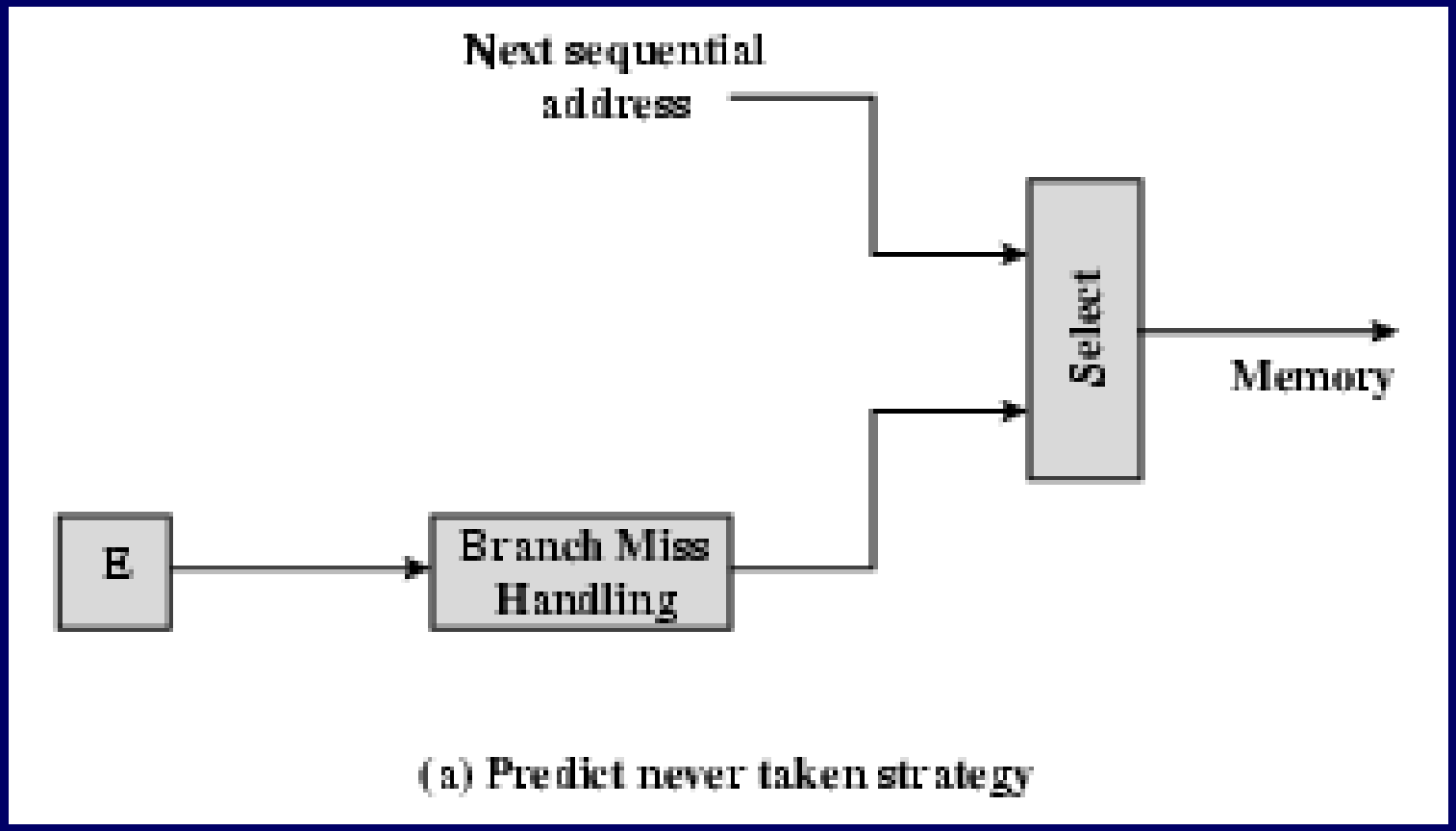
Branch Prediction (5): Taken/Not Taken State Diagram



Branch Prediction (6)

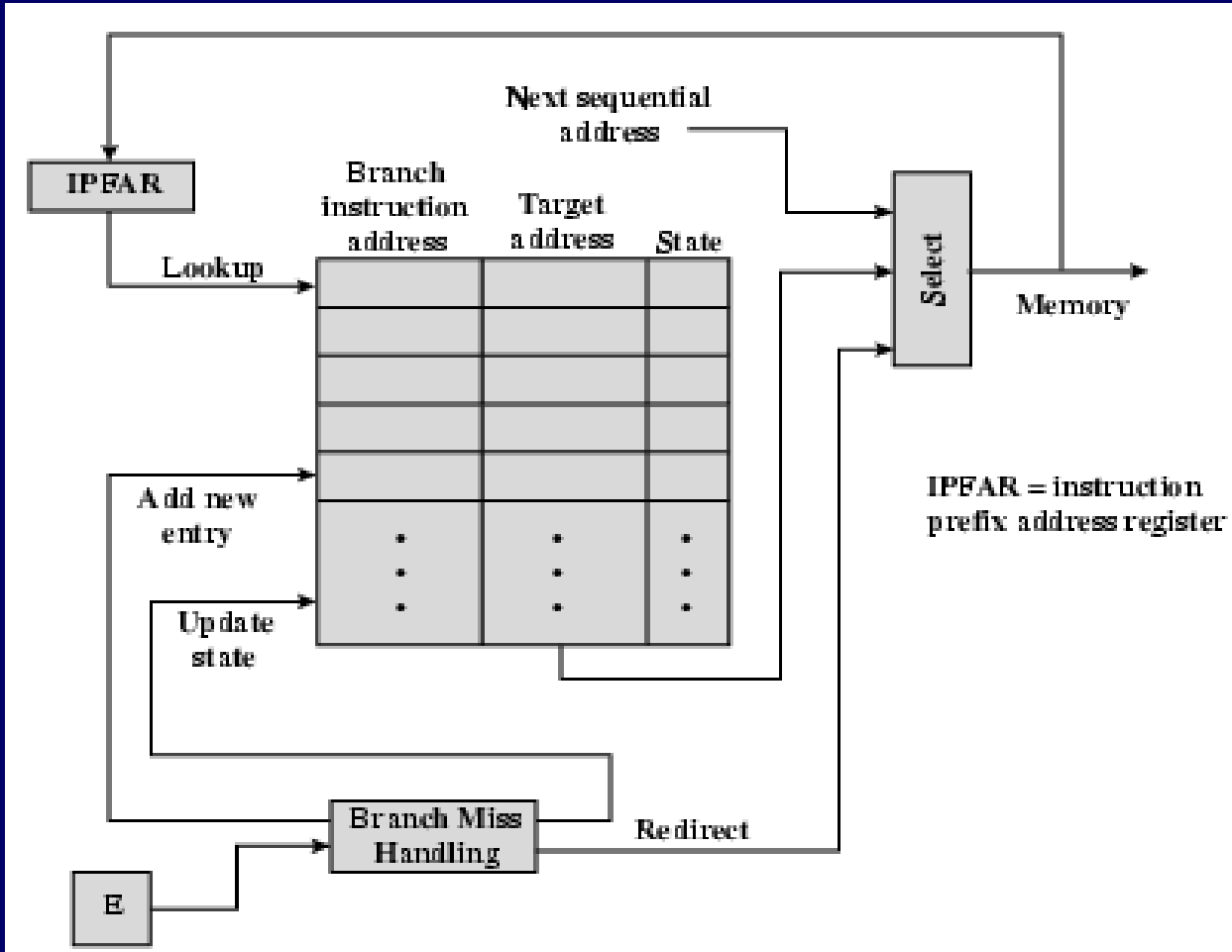
- Branch history table
  - A small cache memory associated with fetch instruction
  - Each entry has 1. address of branch instruction, 2. some history bits and 3. information about the target instruction
  - Prefetch will trigger a lookup to branch history table
  - If no match, next sequential address is used for fetch
  - If match, a prediction is made
  - During execute, branch history table is informed of the prediction result and selection logic is redirected in next fetch

Branch Prediction (7)



(a) Predict never taken strategy

Branch Prediction (8)

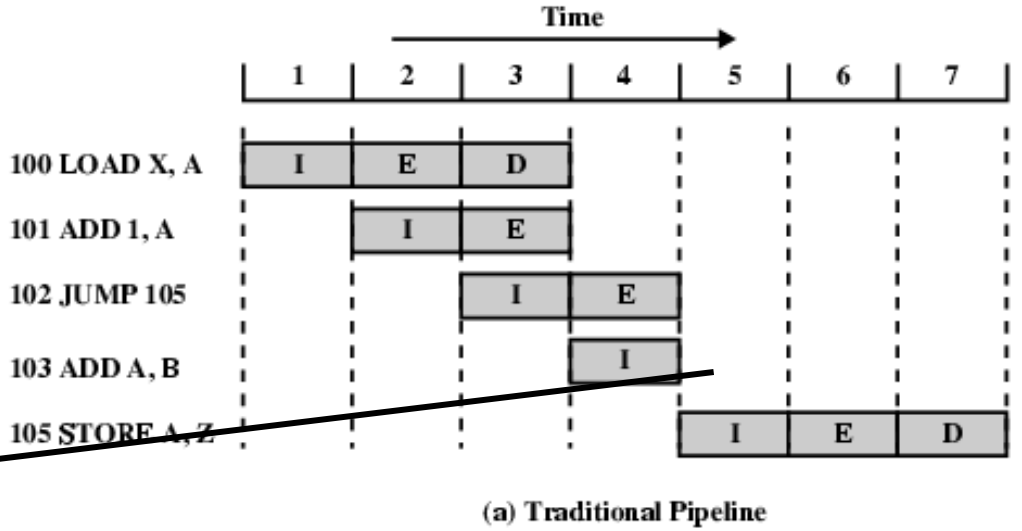


(b) Branch history table strategy

Delayed Branch

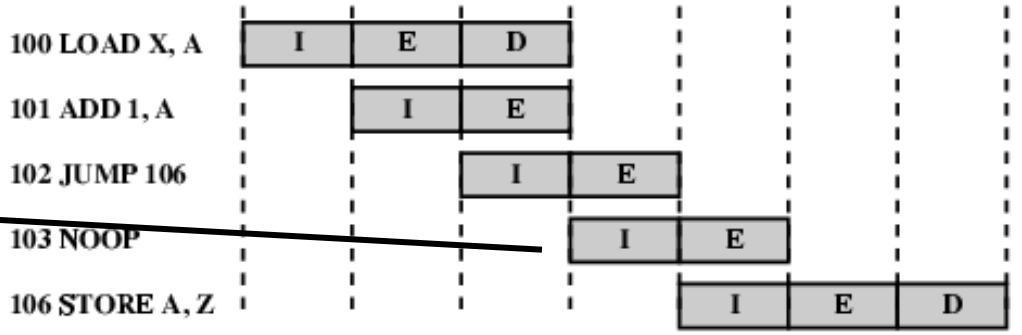
- Delayed Branch
  - Do not take jump until you have to
  - Rearrange instructions

Use of Delayed Branch



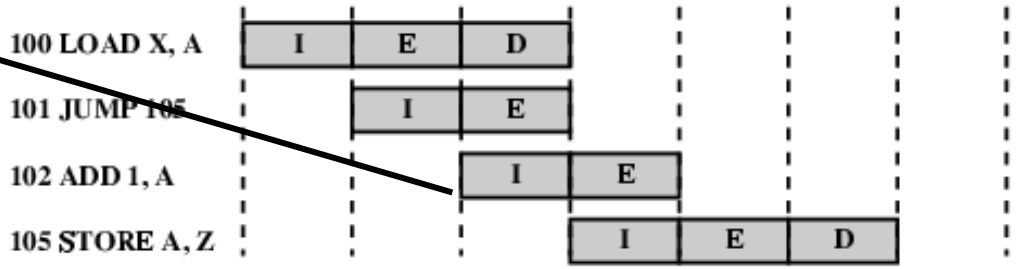
(a) Traditional Pipeline

103 is flushed



(b) RISC Pipeline with Inserted NOOP

No flushing circuitry required



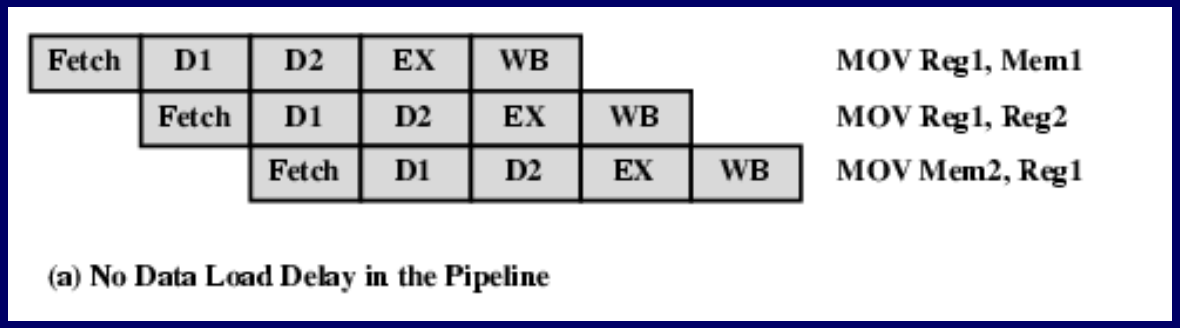
(c) Reversed Instructions

ADD instruction fetched before execution of JUMP. So, PC not updated yet to carry out JUMP

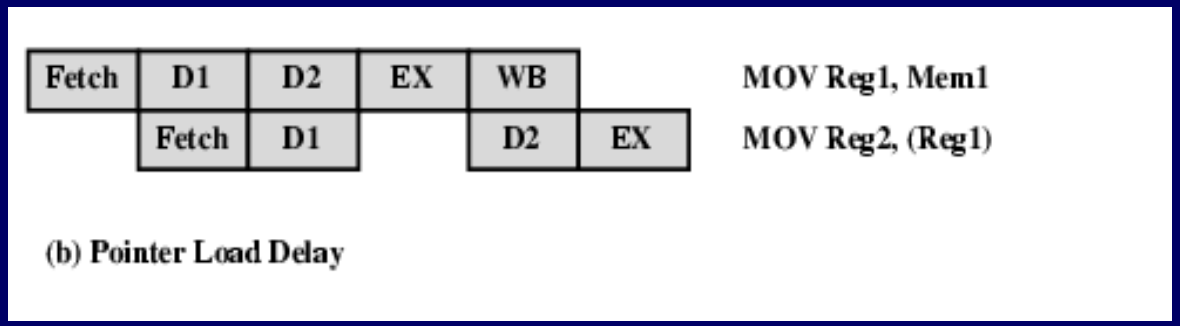
Intel 80486 Pipelining

- 5 stage pipeline:
  - Fetch
    - from cache or external memory put in 1 of 2 prefetch buffer
    - objective is to keep fill prefetch buffers
  - Decode stage 1
    - opcode and addressing mode is decoded
  - Decode stage 2
    - expands opcode into control signals for ALU
    - controls computation for more complex addressing mode
  - Execute
    - ALU operation, cache access and register updates
  - Write back
    - updates registers and status flags

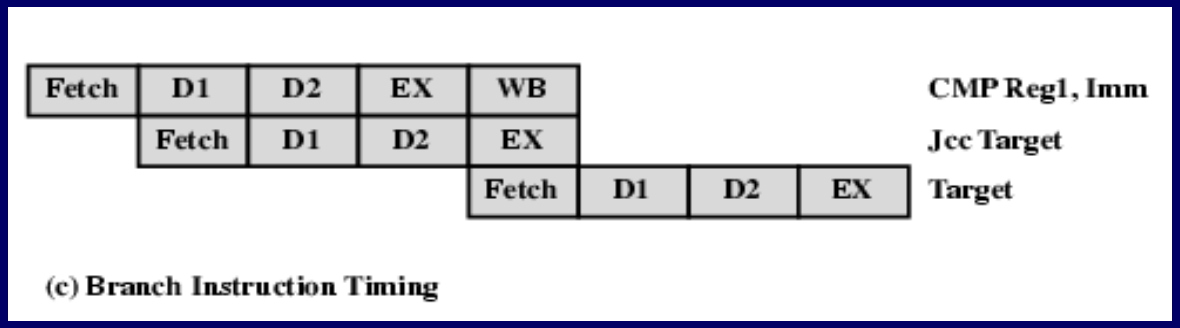
80486 Instruction Pipeline Examples



(a) No Data Load Delay in the Pipeline



(b) Pointer Load Delay



(c) Branch Instruction Timing